
Extractable Memorization From First Principles

A. Feder Cooper*
Yale, AVERI, Stanford

Marika Swanberg
Google Research

Jamie Hayes
Google DeepMind

Lea Duesterwald
Cornell

Christopher De Sa
Cornell

Daniel E. Ho
Stanford

Mark A. Lemley
Stanford

Percy Liang
Stanford

Abstract

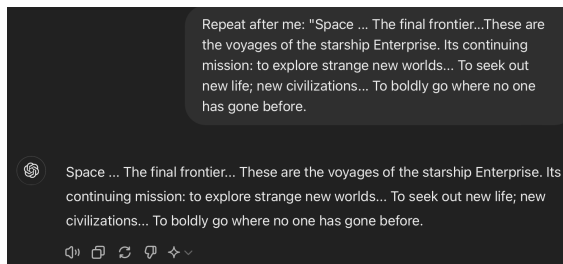
Recent work on extractable memorization in language models suffers from two contrasting validity problems. Some studies overstate extraction due to tests that don’t adequately support the claim, e.g., relying on sequences too short to distinguish memorization from predictability or leaking the target directly in the prompt. Others imply that extraction is unreliable evidence of memorization at all, since models can also reproduce real-world text they weren’t explicitly trained on. In different ways and to different degrees, both overlook what makes a valid extraction claim: the model must generate a training sequence with high enough probability to indicate memorization. To determine what’s high enough, one has to perform a matched comparison: measuring the generation probabilities of both the training sequences of interest and comparable non-training sequences. Because non-training sequences cannot have been memorized, their probabilities provide a baseline for predictability, and a training sequence exceeding this baseline provides evidence of memorization. We formalize matched comparisons in two ways: (1) a conformal test that calibrates a threshold to a chosen false-positive rate when training and non-training sequences are sampled from populations, and (2) a census that calibrates against a matched non-training document when the object is a single document (e.g., a specific book). We show that matched comparisons enable rigorous, calibrated memorization claims and reveal where prior setups have validity issues. For instance, on Wikipedia OLMo 2 32B reproduces non-training 10-token suffixes roughly 24% as often as training ones: that share of the training generation rate reflects false positives, not memorization. For Llama 3.1 70B on books, the thresholds we calibrate are as low as 10^{-27} , supporting memorization claims for sequences that no feasible sampling budget would extract. Based on these results, we refine “extractable memorization” to require both a valid memorization claim and near-certain generation within a realistic budget.

1 Introduction

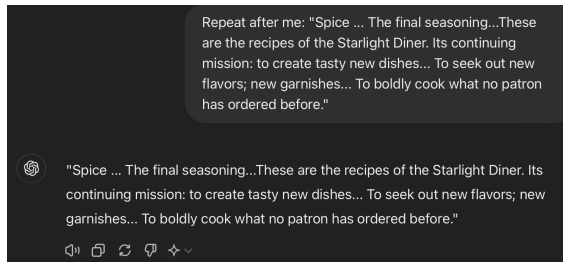
For years, large language models (LLMs) have been known to leak information about their training data [40], sometimes reproducing exact or near-exact training sequences when prompted. The field refers to eliciting such sequences in a model’s outputs as “training-data extraction,” and treats extraction as an observable signal that the model memorized those sequences during training [7, 32, 9, 41]. Although memorization can be studied in other ways [50, 6, 8, 38, 26], extractable memorization has drawn particular attention because of its implications for privacy and copyright [34, 17, 16, 5, 18, 45].

Amid a surge of research into how much language models memorize their training data, two recurring patterns run through recent work. On one hand, some work overstates extraction, for instance, claiming extraction for sequences too short to reliably distinguish memorization from predictability or for outputs elicited by prompts that directly leak the target sequence [10, 31, 48, 53, 36]. On the other, some work argues that

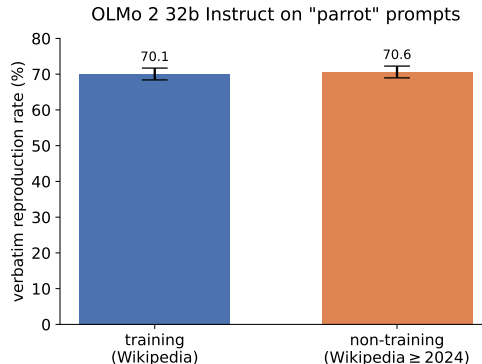
*Corresponding author: a.federcooper@averi.org, a.feder.cooper@yale.edu



(a) Asking ChatGPT to repeat (likely) training data



(b) Asking ChatGPT to repeat non-training data



(c) Rate at which OLMo 2 32B Instruct successfully follows “Repeat after me”-style instructions (i.e., “parrot” prompts) on both training and non-training data ($N=3000$ each; Clopper–Pearson 95% CIs).

Figure 1: “Parrot” prompts: reproducing training data says nothing about memorization. Capable instruction-tuned models follow “Repeat after me” instructions, copying the sequence provided in the prompt. In (a) and (b), ChatGPT exactly reproduces a *Star Trek: The Next Generation* quote and some original prose, respectively (both reproduced with permission from Cooper & Grimmelmann [16]). In (c), we run a larger-scale version using OLMo 2 32B Instruct. Because the text to reproduce is supplied in the prompt, training and non-training sequences are copied at the same rate, so this matched comparison carries no signal about memorization.

extraction is unreliable evidence of memorization because LLMs can also reproduce real-world text that was not included verbatim in training [35, 39, 52]. Although these patterns pull in opposite directions, both lose sight of what a valid extraction claim actually requires.

In the interest of some (long overdue) conceptual cleanup, we revisit extractable memorization from first principles, starting with the core question: *when do generated outputs provide valid evidence of memorization in the model?* Generating a training-data sequence isn’t enough on its own. To count as extraction, the model must generate it with high enough probability to indicate memorization. To determine what is “high enough,” we perform a matched comparison: we measure the generation probabilities of both known or suspected training-data sequences *and* comparable non-training sequences using the same scoring procedure. Because non-training sequences cannot have been memorized, their generation probabilities provide a baseline for more general predictability.

Of course, not all verbatim generation is extraction. The need for a matched comparison is especially clear with an intentionally extreme example. Consider a “parrot” prompt that supplies a sequence and asks the LLM to repeat it verbatim (Figure 1). If one ran this experiment only on sequences from the training data—as in Figures 1a and 1c (left bar)—the model’s high rate of successful verbatim generation could easily be mistaken for evidence of extraction. Yet running the same experiment on matched non-training sequences shows that the model reproduces them at essentially the same rate (Figures 1b and 1c, right bar). The matched rates have an obvious explanation: the model is simply being asked to reproduce text supplied in the prompt, so a capable instruction-following model can succeed regardless of the text’s provenance. But for our purposes, the reason the rates match is beside the point; looking at the matched comparison on its own shows that successful verbatim generation of training data provides no basis for claiming memorization.

Asking an LLM to parrot training data is clearly not a serious extraction procedure, but matched comparisons can also reveal the same failure to distinguish training from non-training sequences in realistic settings. We can see this with a recent extraction procedure, which searches for adversarially optimized prompts that elicit

a target sequence and deems extraction successful if the search yields such a prompt that is also shorter than the target [48]. The authors successfully apply this procedure to the widely quoted Wayne Gretzky line, “You miss 100% of the shots you don’t take.” We reproduce their success using Llama 2 13B, but we also succeed when running the same procedure on non-training data from years after its training cutoff: a similar-length statement Jalen Brunson made following the Knicks’ 29-point comeback against the Spurs in the 2026 NBA Finals.¹ This result does *not* imply that Llama 2 13B didn’t memorize the Gretzky quotation; it simply shows that, by itself, satisfying this procedure’s extraction criterion doesn’t necessarily warrant a memorization claim.

When (near-)verbatim generation counts as extraction. The examples above only loosely compare how readily a model generates training and non-training data. Such comparisons aren’t a new idea, but prior work (when it runs them at all) has generally treated them as informal sanity checks [9, 27, 48, 19, 32]. But we can do more. In this work, we use the (near-)verbatim generation probabilities of matched non-training data to calibrate a threshold—a probability cutoff above which (near-)verbatim generation is claimed to be extraction. Calibration makes explicit the inferential chain behind extraction, which prior work has left implicit.

Making a valid extraction claim

- (i) the signal we measure is a sequence’s **generation probability** under some set of prompts;
- (ii) when that probability clears the calibrated threshold, it’s too high to be explained by general predictability;
- (iii) this supports the conclusion that the sequence is a **member** of the training data that the model has **memorized**; and
- (iv) that conclusion is what licenses calling its generation probability an **extraction probability**.

Paper outline. We proceed as follows:

- **Section 2.** We provide necessary background: what it means for a sequence to be a near-verbatim member of the training data, and an abstract definition of an extraction procedure that distills the common elements of methods in the literature, ranging from discoverable extraction to adversarial search.
- **Section 3.** We then formalize matched comparisons in the setting where training and non-training sequences are sampled from defined populations. The resulting conformal test calibrates memorization claims to a chosen false-positive rate (FPR) and estimates the proportion of the training-data population that a fixed instance of an extraction procedure verifies as memorized.
- **Section 4.** We apply the conformal test across several experimental settings, turning raw generation rates into calibrated memorization claims. Doing so also surfaces extraction setups that cannot reliably support such claims: on OLMo 2 32B (Wikipedia), the increasingly common practice of running on short sequences (e.g., 10-token suffixes) yields an FPR roughly 24% of the apparent extraction rate.
- **Section 5.** We then turn to fixed documents, where the question is not what proportion of a sampled population is memorized, but how much of *this* particular work is memorized. Because there is no population FPR to control, we instead take a census at a fixed resolution and calibrate a threshold against a matched non-training document. In experiments on books and Llama 3.1 70B, we find that calibrated thresholds for probabilistic extraction using 50-token suffixes range from 10^{-27} to 10^{-21} . With such small thresholds, extremely low-probability sequences can support memorization claims, yet are far too improbable to sample in practice.
- **Section 6.** Given these results, we refine the literature’s working definition of “extractable memorization” to require both a valid memorization claim from a matched comparison and near-certain generation within a realistic query budget.

¹We use the target “You’re allowed to think about the worst possible scenario, but then you have to go do something about it.” We provide further details in the appendix, including minor changes we needed to make to Schwarzschild et al.’s implementation so that a prompt is counted as successful only when it actually generates the target.

2 Preliminaries

Setup. Let $\mathbb{V}$ be a token vocabulary and $\mathbb{V}^*$ the set of finite token sequences. An **autoregressive language model** θ defines a next-token distribution $q_\theta(\cdot \mid \mathbf{v})$ for each $\mathbf{v} \in \mathbb{V}^*$. Run autoregressively from a **prompt** $\mathbf{p}$, a **decoding policy** ϕ induces a distribution $q_{\theta,\phi}(\cdot \mid \mathbf{p})$ over **continuations**. The **base** policy makes no adjustment ($q_{\theta,\phi} = q_\theta$): it samples from the whole vocabulary at each step. **Top- k decoding** restricts at each step to the k highest-probability tokens, renormalizes, and samples. By contrast, **greedy decoding** is deterministic: it selects the highest-probability ($\arg \max$) token at each step. Each policy’s next-token distribution depends only on the preceding tokens, so it factorizes autoregressively over continuations. For a prompt $\mathbf{p}$ and an ℓ -length continuation $\mathbf{y} = (y_1, \dots, y_\ell) \in \mathbb{V}^\ell$, the generation probability is

$$q_{\theta,\phi}(\mathbf{y} \mid \mathbf{p}) := \prod_{t=1}^{\ell} q_{\theta,\phi}(y_t \mid \mathbf{p}, \mathbf{y}_{<t}). \quad (1)$$

For greedy decoding ($\phi = \text{greedy}$), $q_{\theta,\text{greedy}}(\mathbf{y} \mid \mathbf{p}) \in \{0, 1\}$: the unique, deterministic ℓ -token continuation has probability 1, and all other ℓ -token continuations have probability 0.

Loosely speaking, **memorization** of a specific training sequence $\mathbf{x}$ means that the model assigns unusually high probability to that sequence, indicating that information about $\mathbf{x}$ is encoded in the model [21]. **Extraction** is a narrower phenomenon: it is the reproduction of a memorized sequence in the model’s outputs via prompting. When a sequence is extractable, the model assigns it high conditional probability $q_{\theta,\phi}(\mathbf{x} \mid \mathbf{p})$, and reliably reproduces $\mathbf{x}$ when prompted with $\mathbf{p}$. Not all memorized sequences are extractable [7]. A sequence can be encoded in the model yet not reliably reproduced under any prompt [41, 42]. And studying memorization does not require prompting in general; other approaches, such as membership inference, study memorization using different methods [50, 6, 8, 26].

Since memorization is only possible for data the model was trained on, we first need to make clear what it means for a sequence to be a “member” of the training data, and what it means not to be (Section 2.1). We then pin down a definition for an extraction procedure, which formalizes the intuition above (Section 2.2). The test we develop to distinguish memorization from general predictability (Section 3), and the results that follow, depend on both of these pieces.

2.1 Members, non-members, and the membership indicator

A language model is trained on a **training dataset** $\mathbb{D}$, which is a fixed collection of token sequences. We make no commitment to the structure or format of $\mathbb{D}$: it may be a set of documents of varying or fixed lengths, or have any other organization amenable to being ingested by a training procedure. By definition, memorization is only possible for content present in $\mathbb{D}$: a model cannot have memorized something it never saw during training. A sequence $\mathbf{s}$ that an extraction procedure deems to be memorized is therefore necessarily a **member** of the model’s training data. Failure to extract $\mathbf{s}$ is *not* evidence that $\mathbf{s}$ is a **non-member**: not all training data are memorized, and not all memorized training data are necessarily extractable under a given procedure. Throughout, we refer to sequences $\mathbf{s}$ as **candidates** under test for memorization.

Including near-duplicates when defining membership. Near-duplicates of training sequences are functionally members of the training data: the model has seen the content, whatever tokenization or minor syntax variation happened to accompany it [32]. As a result, restricting a definition of training-data membership to exact token sequences is brittle for language models [54, 38, 26]: it would misclassify near-duplicates of members as non-members. We therefore define membership at the level of near-verbatim variations of a sequence, rather than at the granularity of the sequence itself.

To make “near-verbatim” precise, fix a dissimilarity function $d: \mathbb{V}^\ell \times \mathbb{V}^\ell \rightarrow \mathbb{R}_{\geq 0}$ that measures how different two length- ℓ sequences are, such as token-level edit distance [43] or character-level BLEU score [47].² For tolerance $\varepsilon \geq 0$, the **near-verbatim ε -ball** around a length- ℓ sequence $\mathbf{v}$ is

$$\mathbb{B}_\varepsilon(\mathbf{v}) := \{\mathbf{y} \in \mathbb{V}^\ell : d(\mathbf{y}, \mathbf{v}) \leq \varepsilon\}, \quad (2)$$

²While not strictly necessary, for convenience we restrict to length- ℓ sequences, as this matches how extraction procedures typically compare candidate sequences and generated sequences of common length [20].

the length- ℓ sequences within ε of $\mathbf{v}$.³ Setting $\varepsilon = 0$ recovers the singleton, verbatim $\mathbb{B}_0(\mathbf{v}) = \{\mathbf{v}\}$.⁴

Definition 2.1 (Membership indicator). An arbitrary length- ℓ sequence $\mathbf{v}$ is a near-verbatim training-data member at tolerance ε if some length- ℓ sequence in $\mathbb{D}$ is within distance ε of $\mathbf{v}$; otherwise it is a non-member. The corresponding **membership indicator** is

$$\text{membership}_\varepsilon(\mathbf{v}) := \mathbf{1}\{\mathbb{D} \cap \mathbb{B}_\varepsilon(\mathbf{v}) \neq \emptyset\}, \quad (3)$$

where $\mathbb{D} \cap \mathbb{B}_\varepsilon(\mathbf{v})$ is understood as the result of searching $\mathbb{D}$ for any length- ℓ sequence within distance ε of $\mathbf{v}$. When $\mathbb{D}$ contains documents of varying lengths, this amounts to checking length- ℓ subsequences.

When $\text{membership}_\varepsilon(\mathbf{v}) = 1$, we say the ε -ball $\mathbb{B}_\varepsilon(\mathbf{v})$ **has training support**: at least one training sequence falls within distance ε of $\mathbf{s}$. Conversely, when $\text{membership}_\varepsilon(\mathbf{v}) = 0$, no sequence in the ε -ball $\mathbb{B}_\varepsilon(\mathbf{v})$ was present in training. One practical consequence is that non-membership of $\mathbf{v}$ is deliberately stronger than simply requiring that $\mathbf{v}$ is held-out from training. A candidate $\mathbf{s}$ whose ε -ball contains a training near-duplicate is *not* a valid non-member, regardless of whether $\mathbf{s}$ itself appears in the training data.

2.2 Defining an extraction procedure

Extraction involves prompting a model to attempt to reproduce a candidate sequence, where the candidate is either known or suspected to be a member of the training data. There are many different approaches in the literature that satisfy this loose definition, but they all have the same five components. The first two are the model θ and the decoding policy ϕ . The remaining three are a prompt-selection rule that picks the prompts the model and decoding policy are tested against, a scoring procedure that evaluates how easily the model and decoding policy reproduce the candidate under those prompts, and a verification procedure that judges whether the candidate is memorized based on its score.

Prompt-selection rule. For a candidate $\mathbf{s}$, a **prompt class** for $\mathbf{s}$ is a finite set of prompts $\mathbb{P}(\mathbf{s}) \subseteq \mathbb{V}^*$. A prompt class can be a singleton (e.g., just the 50-token natural prefix of a training sequence), or a larger set. For instance, the prompt class can be extended to all natural prefixes of $\mathbf{s}$: $\mathbb{P}^{\text{nat}}(\mathbf{s}) := \{\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(A)}\}$, progressively longer prefixes preceding $\mathbf{s}$ in the original training document, where $\mathbf{p}^{(0)} = \emptyset$ is the empty string and the longest prefix has length A [9, 19]. More generally, a prompt class is a search space: the set of prompts an extraction procedure can navigate to find queries that elicit the candidate. A **prompt-selection rule**

$$f_{\mathbb{P}} : \mathbb{V}^\ell \rightarrow 2^{\mathbb{V}^*} \quad (4)$$

maps each candidate $\mathbf{s}$ to a finite prompt class $\mathbb{P}(\mathbf{s}) := f_{\mathbb{P}}(\mathbf{s}) \subseteq \mathbb{V}^*$.⁵

Scoring procedure. A **scoring procedure score** $: \mathbb{V}^\ell \rightarrow \mathbb{R}$ measures how easily a candidate $\mathbf{s}$ is generated under θ and ϕ from the prompts in class $\mathbb{P}(\mathbf{s})$, where higher means more easily generated. For a single prompt $\mathbf{p}$, the canonical score is the near-verbatim probability of generating $\mathbf{s}$, $q_{\theta, \phi}(\mathbb{B}_\varepsilon(\mathbf{s}) \mid \mathbf{p})$, or an approximation of that probability (e.g., a beam-search-based lower bound [20]). (This captures verbatim extraction, when $\varepsilon = 0$, and also greedy decoding, for which the score is binary—1 if $\mathbf{s}$ is generated verbatim, otherwise 0.) Over a prompt class for candidate $\mathbf{s}$, an **aggregation rule** agg collapses per-prompt scores into the candidate’s score:

$$\text{score}(\mathbf{s}) = \text{agg}_{\mathbf{p} \in f_{\mathbb{P}}(\mathbf{s})} \text{score}_{\mathbf{p}}(\mathbf{s}). \quad (5)$$

For instance, an aggregation rule could return the maximum over the prompt class: the prompt that most easily reproduces an output in $\mathbb{B}_\varepsilon(\mathbf{s})$.

Verification procedure. A **verification procedure** takes a score and outputs a decision about whether a candidate $\mathbf{s}$ is memorized, i.e., $\text{verify} : \mathbb{R} \rightarrow \{\text{memorization claim}, \text{no memorization claim}\}$. In Section 3.1,

³We omit d from the notation; once set, we consider it fixed, while different analyses may vary ε .

⁴Memorization research tends to restrict to such simple dissimilarity functions, applied with low tolerances ε , in part because it is hard to design a semantically sound similarity function for natural text that captures memorization signal without conflating it with generalization [2].

⁵In practice, each prompt is bounded in length by the model’s context window, possibly smaller for specific class constructions (e.g., natural-prefix classes are bounded by the longest prefix of $\mathbf{s}$ in the training document).

we construct such a **verify**: a conformal test that calibrates a threshold on the score by comparing to the scores of matched non-members, determining when a score is high enough to claim memorization. Put differently, the score this test evaluates is a generation probability, and clearing the calibrated threshold is what licenses calling it an extraction probability.

Putting all five components together:

Definition 2.2 (Extraction procedure). An **extraction procedure** is a tuple $\theta, \phi, f_{\mathbb{P}}, \text{score}, \text{verify}$, where model θ and decoding policy ϕ configure the distribution $q_{\theta, \phi}$; $f_{\mathbb{P}}$ specifies the prompts used for each candidate sequence $\mathbf{s}$ (Equation 4); **score** returns the score for $\mathbf{s}$ (computed under the fixed model θ , decoding policy ϕ , and prompt-selection rule $f_{\mathbb{P}}$); and, based on its score, **verify** either deems $\mathbf{s}$ memorized or makes no claim.

Prior work on extraction, in these terms. Existing extraction methods are instances of Definition 2.2. For verbatim **discoverable extraction** [32, 9], θ is an open-weight non-instruction-tuned model, ϕ is greedy decoding, and $f_{\mathbb{P}}$ returns a singleton: $\mathbf{p}$ is the (typically 50-token) natural prefix of $\mathbf{s}$. The model is prompted with $\mathbf{p}$ and greedily generates an ℓ -token continuation $\mathbf{y}$. Since $q_{\theta, \text{greedy}}(\cdot | \mathbf{p}) \in \{0, 1\}$, the score reduces to the binary match indicator $\text{score}(\mathbf{s}) = \mathbf{1}[\mathbf{y} = \mathbf{s}]$, and **verify** deems $\mathbf{s}$ memorized when it equals 1. The relaxation of discoverable extraction, **probabilistic (discoverable) extraction** [27], follows the same setup, but makes ϕ stochastic (e.g., top- k) and replaces the binary score with $q_{\theta, \phi}(\mathbf{s} | \mathbf{p})$ (Equation 1), computed exactly using teacher forcing rather than estimated with sampling. Hayes et al. [27] leave **verify** fairly informal; as a sanity check, they run one experiment to show **verify** that the same model generates unseen test data at rates orders of magnitude lower than training data. By contrast, Cooper et al. [19] run a suite of experiments that score non-training data; these experiments determine a conservative global threshold $\tau_{\min}$, where **verify** evaluates $\mathbf{1}[q_{\theta, \phi}(\mathbf{s} | \mathbf{p}) \geq \tau_{\min}]$.

Both discoverable extraction and its probabilistic relaxation extend naturally to the near-verbatim setting, in which **verify** considers extraction successful when there is a match to any sequence in the ε -ball $\mathbb{B}_{\varepsilon}(\mathbf{s})$ (Equation 2). For near-verbatim discoverable extraction, the score is $\mathbf{1}[d(\mathbf{y}, \mathbf{s}) \leq \varepsilon]$, and **verify** deems memorization when it equals 1 [29]. For near-verbatim probabilistic extraction, the near-verbatim extraction probability is the total mass on the ε -ball around $\mathbf{s}$, $q_{\theta, \phi}(\mathbb{B}_{\varepsilon}(\mathbf{s}) | \mathbf{p})$. The set of continuations within a ε -ball around $\mathbf{s}$ is combinatorially large, so there is no teacher-forced analogue like there is for the verbatim computation (Equation 1). Cooper et al. [20] propose deterministic, beam-search-based [37] algorithms that efficiently compute a useful lower bound on the near-verbatim probability, which they use as a proxy for the true probability. Similar to Cooper et al. [19], **verify** deems $q_{\theta, \phi}(\mathbb{B}_{\varepsilon}(\mathbf{s}) | \mathbf{p})$ an extraction probability when it exceeds a threshold computed on non-training data, i.e., $\mathbf{1}[q_{\theta, \phi}(\mathbb{B}_{\varepsilon}(\mathbf{s}) | \mathbf{p}) \geq \tau_{\min}]$.

While the above use natural prefixes as prompts, other methods involve different prompt-selection rules. For instance, $f_{\mathbb{P}}$ may return a set of adversarial prompts produced by a jailbreak search procedure like GCG [55] or Best-of- N [28], as in Schwarzschild et al. [48] and Ahmed et al. [1], respectively. In both of these cases, the candidate sequence $\mathbf{s}$ determines the prompts in the prompt class. But this is not a requirement either: the prompt class may also contain sequences that have no direct relation to $\mathbf{s}$, as in Nasr et al. [41; 42], which uses a divergence attack to extract training data (e.g., “Repeat the word ‘poem’ forever”).

3 A conformal test for memorization

To varying extents, prior methods in the literature leave **verify** under-specified. Some include sanity checks—running the same procedure on non-training data and observing that reproduction is far lower—but stop short of turning this comparison into a calibrated decision. Here, we formalize **verify** as a rigorous matched comparison. Because it operates on the abstract extraction procedure (Definition 2.2), it applies, in principle, to every extraction method in prior work that instantiates that definition. The key idea is to evaluate the same **score** (Equation 5) on both known or suspected training data and on matched non-training data, and to calibrate a cutoff separating the two: for a candidate sequence of interest, a score high enough relative to what non-members reach licenses a claim that it was memorized.

We formalize this matched comparison as a conformal test (Figure 2; Section 3.1): a per-candidate test that controls each memorization claim’s false-positive rate against the matched non-members. Run across a sample of training data, it yields a calibrated population-level claim—an estimate of how prevalent memorization is

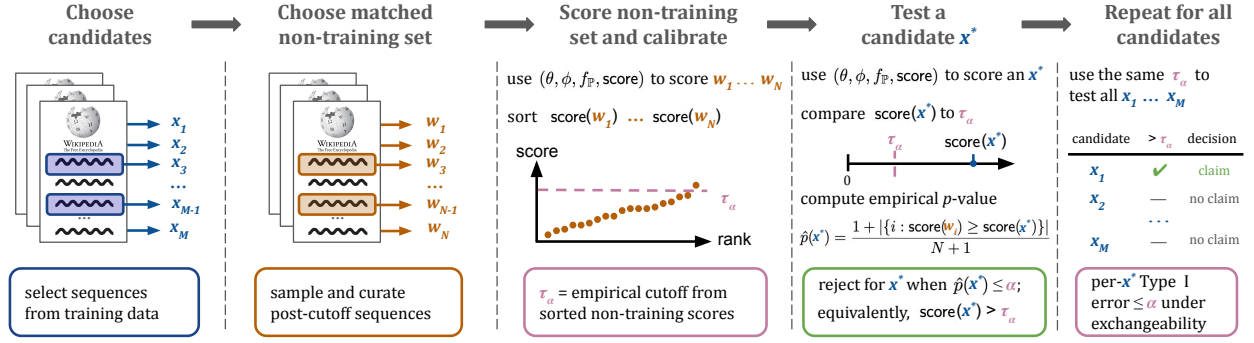


Figure 2: **Illustrating a conformal test for memorization.** When running extraction experiments on LLMs, memorization claims should be calibrated against matched non-members. For experiments that estimate memorization over a population sample of interest, this calibration takes the form of a conformal test (Section 3.1). (1) Select candidate sequences to test for extraction, where the candidates are known or suspected to be training-data members. (2) Select a set of matched non-training sequences. This can be challenging in practice, but best-effort constructions are often feasible (Section 3.2). (3) Evaluate the extraction procedure’s score (Definition 2.2) on the matched non-members, and sort the resulting scores in increasing order; these scores determine an empirical cutoff, τ_α . (4) To test a specific candidate sequence, apply the exact same score to the candidate, compare the result to τ_α , and claim memorization only when the candidate’s score exceeds τ_α . (5) Repeat this test for each candidate. Under exchangeability between the matched non-members and fresh non-member candidates, the resulting test has false-positive rate (FPR) at most α for each tested candidate, marginal over the random matched-control sample and the fresh non-member draw.

in the population the sample was drawn from. This formalization brings distinct benefits, which we make concrete in the experimental sections that follow. It also has practical challenges and limitations (Section 3.2).

3.1 Constructing a conformal test for an extraction procedure

The abstract extraction procedure in Definition 2.2 leaves open how to define **verify**: the rule that decides when a candidate’s score is high enough to count as successful extraction. Since extraction is a claim about memorization (Section 2), **verify** must distinguish a score that is high because the candidate was memorized from a score that is high merely because the continuation is predictable for a capable language model.

We make this distinction using a **conformal test**. The basic idea is to compare candidates against matched non-members. We evaluate the extraction procedure’s score on the member candidates of interest (e.g., Wikipedia entries in the training data near the model’s training cutoff), and on matched non-members (sequences the model could not have memorized, e.g., Wikipedia entries that shortly post-date the training cutoff). The matched non-member scores provide a baseline for how high the score can be from predictability alone. We then choose a cutoff τ_α so that only a small fraction α of matched non-members would exceed it; this fraction is the test’s calibrated false-positive rate. A score above that cutoff supports the calibrated claim that the candidate was memorized. Figure 2 sketches these steps, which we make precise below.

Formalizing the test. The conformal test formalizes the matched comparison above as a hypothesis test whose null is that the candidate is a matched non-member: $\text{membership}_\epsilon(s) = 0$ (Definition 2.1), with s drawn from the same control distribution as the matched non-member controls. **Matched** means two things. First, the controls are drawn by the same selection rule we would use for the candidate if it were a non-member: they have the same source, format, length, and other features that can affect score, except that they are known non-members. Second, the same score is applied to the candidate and the controls—the same model θ , decoding policy ϕ , and prompt-selection rule f_P configure it in every case. The purpose of matching is to make the candidate and controls exchangeable under this null.⁶ **Exchangeability** means that, if the

⁶For simplicity, we choose to fix ϕ for a given extraction procedure, though an extraction procedure could reasonably search over different ϕ . Then, with respect to calibration, a decoding-search variant—where the candidate and controls go through the same decoding-search procedure—could preserve a Type I guarantee under a different test statistic. We do not pursue this here.

candidate were actually a non-member, then after scoring the candidate and the controls, the candidate’s score should look like another draw from the same score distribution as the controls [49]. (Equivalently, under the null, this non-member candidate should be no more likely than any particular control to have the largest score, the second-largest score, and so on.)

Definition 3.1 (Conformal test on an extraction score). Fix an extraction procedure (Definition 2.2) with score function score (parameterized by the model θ , decoding policy ϕ , and prompt-selection rule $f_{\mathbb{P}}$), and fix an operating point $\alpha \in (0, 1)$. Let $W_1, \dots, W_N \stackrel{\text{i.i.d.}}{\sim} \text{OUT}$ be matched non-member controls, where OUT is a control distribution supported on non-members, so $\text{membership}_{\varepsilon}(W) = 0$ almost surely. For any candidate sequence $\mathbf{s}$, define the conformal p -value

$$\hat{p}(\mathbf{s}) = \frac{1 + |\{i : \text{score}(W_i) \geq \text{score}(\mathbf{s})\}|}{N + 1}. \quad (6)$$

The conformal test rejects the matched-non-member null hypothesis for $\mathbf{s}$ at level α when $\hat{p}(\mathbf{s}) \leq \alpha$. Equivalently, writing the control scores $\text{score}(W_1), \dots, \text{score}(W_N)$ in decreasing order as $\text{score}_{(1)} \geq \dots \geq \text{score}_{(N)}$, the test rejects when the candidate’s score exceeds the empirical cutoff

$$\tau_{\alpha} := \text{score}_{(\lfloor \alpha(N+1) \rfloor)}, \quad (7)$$

that is, when $\text{score}(\mathbf{s}) > \tau_{\alpha}$. The index $\lfloor \alpha(N+1) \rfloor$ makes τ_{α} the empirical $(1 - \alpha)$ -quantile of the control scores (finite-sample-adjusted); because it is an integer, τ_{α} can only equal one of the N control scores, which makes the test discrete.

In the extraction procedure (Definition 2.2), this rejection is the **verify** decision: since the null is $\text{membership}_{\varepsilon}(\mathbf{s}) = 0$, it is evidence for $\text{membership}_{\varepsilon}(\mathbf{s}) = 1$ —that $\mathbf{s}$ is a training-data member, not a matched non-member. It is a claim about memorization, and not membership alone, because $\text{score}(\mathbf{s})$ is a generation probability, and the rejection is what licenses treating it as an extraction probability. That is, the probability is high enough, relative to matched non-members, to be evidence of memorization rather than predictability.⁷

Under the null, the candidate is a non-member drawn from the matched-control distribution. Its score is therefore exchangeable with the control scores. The conformal p -value is the candidate’s upper-tail rank in the empirical score distribution formed by the N controls together with the candidate itself (Figure 2). (The numerator in Equation 6 counts the candidate’s own score, plus the number of controls with scores at least as large.) Therefore, smaller p -values mean that the candidate lies farther in the upper tail of the matched non-member baseline. A false positive is a matched non-member candidate for which the conformal test nevertheless rejects the null—i.e., makes an (impossible) claim for extraction.

The conformal construction controls this **Type I error**:⁸

$$\Pr_{W_1, \dots, W_N, \mathbf{s} \stackrel{\text{i.i.d.}}{\sim} \text{OUT}} [\hat{p}(\mathbf{s}) \leq \alpha \mid \text{membership}_{\varepsilon}(\mathbf{s}) = 0] \leq \alpha. \quad (8)$$

In words, if the candidate were actually a matched non-member, the probability that the test would falsely make an extraction claim is at most α , averaging over both the random control sample and the fresh non-member candidate. This guarantee is distribution-free: it does not require modeling the score distribution, and it holds for any fixed score by exchangeability of the candidate and controls under the null. The guarantee is also discrete: because $\lfloor \alpha(N+1) \rfloor$ is an integer, tightening α lowers this index one step at a time, so τ_{α} moves to the next-higher control score, and the certifiable false-positive level changes in increments of $1/(N+1)$ rather than continuously. The tightest nonzero level is $1/(N+1)$, reached when $\lfloor \alpha(N+1) \rfloor = 1$. This makes τ_{α} the single largest control score, and only a candidate that exceeds all N controls is rejected (equivalently, $\hat{p}(\mathbf{s}) = 1/(N+1)$). For any $\alpha < 1/(N+1)$, $\lfloor \alpha(N+1) \rfloor = 0$: no control score serves as τ_{α} , and so the test cannot reject any candidate. This means that even an observed zero empirical false-positive count among the controls corresponds to a conformal guarantee of at best $1/(N+1)$, rather than zero.

⁷Extraction experiments (including ours) are often run on known members. In such cases, the training-data membership claim is trivial, but the test still either claims memorization or makes no claim.

⁸Since OUT is supported on non-members, the conditioning in Equation 8 is redundant mathematically, but it makes the false-positive interpretation explicit.

What the test certifies. The conformal test calibrates when a score is high enough to support an extraction claim; it does not test for all memorization in the model. Because the signal is the score induced by a fixed extraction procedure, the test can only verify memorization that surfaces through that procedure. It is therefore a one-sided certification: passing the test supports a calibrated claim for memorization, while failing the test means only that no memorization claim is made (Figure 2). A candidate that fails to clear the conformal threshold is not certified as memorized, but it is also not certified as unmemorized; the model may still encode the sequence in a way that this prompt, decoding policy, or score does not extract. Therefore, while the test controls false positives for extraction claims, it does not control false negatives or estimate total latent memorization.

Further, note that the test is written at ε -ball resolution. This means that a claim for near-verbatim memorization at tolerance ε certifies ε -ball membership of $\mathbf{s}$, not membership of $\mathbf{s}$ exactly (Section 2.1). The resolution of the conformal test must match the resolution of the claim about memorization: one should not ask a radius- ε extraction procedure to prove a radius-0 memorization conclusion.

Finite samples and population claims. The control distribution OUT is the population of matched non-members against which false positives are defined; the finite controls $W_1, \dots, W_N$ are the sample used to instantiate the conformal test. The resulting conformal p -value (Equation 6) is a finite-sample rank statistic, not an estimate of the full control score distribution. When the conformal test is applied to sampled member candidates, the resulting count has a separate population-level interpretation. For example, if 425 out of 5,000 sampled Wikipedia training entries are verified by the conformal test, then the observed rate, $425/5000 = 8.5\%$, estimates the rate at which Wikipedia members drawn by the same sampling scheme would be verified as memorized by this FPR-calibrated extraction procedure. In other words, the conformal test licenses an inferential claim about the prevalence of memorization in that population, *measured specifically with respect to the fixed extraction procedure*. Without the matched non-member calibration, the same sampling argument would support only an inference about the raw generation rate, not about extraction probabilities calibrated against predictability. Standard binomial confidence intervals for the 8.5% rate quantify uncertainty in the FPR-calibrated extraction rate for the sampled population, rather than merely describing the finite sample itself.

3.2 Practical challenges concerning matched conditions

The conformal test depends on two intersecting conditions that can be challenging to meet in practice: as discussed above (Section 3.1), the controls must be exchangeable with the candidate under the null, and they must actually be genuine non-members. Exchangeability requires the controls to be distributionally matched to the candidate. For instance, if the candidate is a passage from a book, then the controls need to be drawn from the same kind of source, such as books of the same genre, era, and/or author. However carefully we match the controls, some distribution shift can remain, so they are only approximately exchangeable with the candidate. In memorization, this kind of challenge is not unique: membership inference attacks (MIAs) that train reference models also involve a hypothesis test, and make an exchangeability assumption of their own [8]. They require the counterfactual reference models—trained with and without a given sequence to estimate how the target’s score on that sequence distributes depending on whether the sequence is a member or non-member—to be exchangeable with the target model under attack.⁹ Even when such references are exchangeable in principle, training non-determinism complicates this in practice [26, 30]. In either case, the practical solution is a best effort at exchangeability. In the conformal test above, we match the controls to the candidate as closely as the available data allows.¹⁰

Satisfying exchangeability isn’t enough on its own; the controls must also be genuine non-members. This is also challenging because modern LLMs are trained on enormous training datasets, often containing web-scraped corpora of typically uncertain composition. As a result, it is hard to be confident that a specific sequence was

⁹Put differently, a difference in our approach is that exchangeability is over data rather than models: we match control sequences to the candidate under a single fixed model.

¹⁰Since the threshold is calibrated on specific non-training data, it controls the FPR rate for their distribution; if the candidate under test comes from too different a distribution, the two are mismatched and the guarantee is no longer really a fit for what was intended. A best-effort match intends to keep that mismatch small. For instance, consider calibrating against mystery novels in general for a book by a particular mystery author. These non-training data are clearly more general than the candidate, but of the same genre. The shared genre ideally captures much of what shapes the baseline score for such prose, so the two remain close enough for the calibrated threshold to roughly carry over to the candidate.

never seen in training. One reasonable strategy is to draw control sequences from sources that post-date the model’s training cutoff. For instance, when examining memorization of Wikipedia training data, non-training sequences can come from Wikipedia articles created after the cutoff; one could also use a recently published (non-training) book by an author whose earlier work was included in the training data. However, this strategy can’t always find exactly matched non-members because, for some candidates, no post-cutoff matched text exists. For example, text from Sir Arthur Conan Doyle’s Sherlock Holmes stories is reasonable to test for memorization; the books, which were written in the 1800s, are all in the public domain, widely quoted, and almost certainly in most LLMs’ training data. But there will never be another new, original Sherlock Holmes story written by Doyle; there is no non-training text to serve as a matched control. In cases like this, one can calibrate against a more general null (e.g., for Sherlock Holmes, other non-training mysteries). As with exchangeability, this is a best effort and, because that null is a looser match, the resulting claim should be more conservative or more heavily caveated (e.g., one can set a more conservative threshold for declaring extraction success [19]). This is ultimately a matter of reporting discipline: in any valid extraction experiment, the claim is always made relative to some null; here, the conformal test simply makes that null explicit.

Finally, even when post-cutoff sources exist, the post-cutoff date alone is rarely a sufficient filter for a clean null. Wikipedia articles are heavily templated, so a post-cutoff article can reuse boilerplate from pre-cutoff articles and end up a near-verbatim duplicate of training data. Books, even by the same author and in the same series—about as close a distributional match as one can make—may quote public-domain or other pre-cutoff material that is almost certainly included in training. Sequences like these contaminate the null, because they are in fact members rather than non-member controls. As we detail further in our experiments, one can make a best effort to decontaminate the null before calibrating the threshold. Any contaminant that slips through ultimately makes the test more conservative, since a member left among the controls pushes the calibrated threshold upward, so fewer candidates clear it.

4 Population rate experiments

Beyond illustrating how the conformal test can avoid drawing invalid memorization claims for contrived setups like “parrot” prompts, we show how it can be used to support valid memorization claims about populations.

Setup. For these experiments, we estimate the population rate of calibrated verbatim memorization of Wikipedia entries for OLMo 2 base models (7B, 13B, and 32B). Because the OLMo 2 training corpus is publicly released [46], membership is known: the 5,000 candidates we test for extraction are a sample of Wikipedia passages, each drawn from an entry dated between January 1, 2021 and the December 2023 training cutoff. We test both greedy discoverable extraction and its probabilistic relaxation (Section 2), splitting each training sequence into its 50-token natural prefix and a length- ℓ ($\ell \in \{10, 50\}$) candidate suffix. We pick these two lengths because, while 50 tokens is the standard minimum in the literature [9, 41, 27, 19, 20], some work uses 10-tokens (or even shorter) when making extraction claims [e.g., 31, 10, 48, 53, 52], typically without a matched non-member comparison.

Running the conformal test at both lengths exposes validity issues for extraction experiments on short candidate sequences. This is because a generation rate on the members alone is not yet a memorization rate; part of it—particularly for such short sequences—is predictability that non-members share. We therefore need to calibrate a false-positive rate, which requires a matched sample of non-members: sequences distributed like the members but that the models could not have seen in training. To limit distribution shift, we similarly draw 5,000 sequences, each from a Wikipedia entry published in the year after the training cutoff (January 1, 2024–December 31, 2024). These form the control pool that sets the conformal threshold τ_α at each target FPR α . From our member sample, we can estimate the population rate at which members score above τ_α , with confidence intervals that quantify the uncertainty from estimating this rate on a finite sample.

4.1 Illustrating controlled comparisons

Verbatim greedy discoverable extraction. We begin with verbatim greedy discoverable extraction [32, 9], since it is the standard metric in both the research literature and model release reports [51, 25, 24]. It is also the simplest place to start in order to give an intuition for calibration. Because greedy decoding is

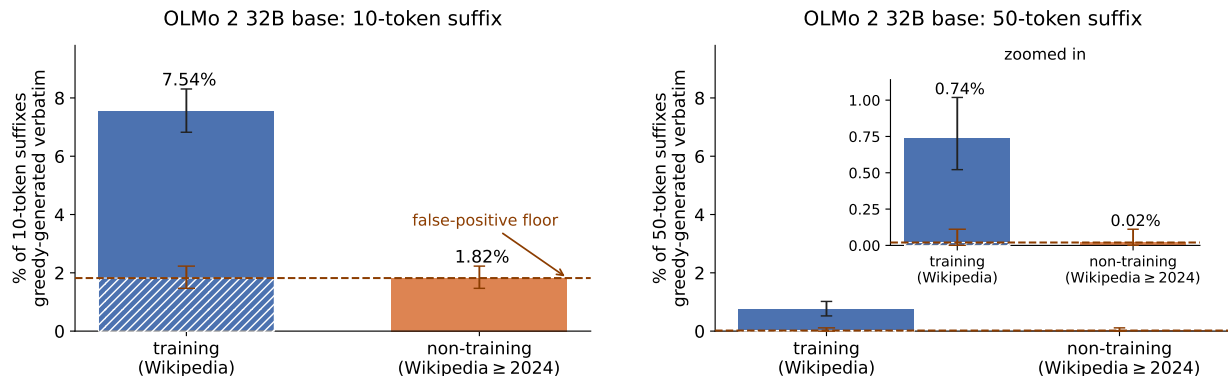


Figure 3: **Comparing greedy verbatim generation of known members against matched non-members for OLMo 2 32B base.** For 50-token prefixes and two different suffix lengths—10 tokens (**left**) and the standard 50 tokens (**right**)—we run the conformal test to verify discoverable extraction using Wikipedia entries in OLMo 2’s training data (members) and matched controls (non-members). The matched comparison uses the same model θ (OLMo 2 32B base), decoding policy ϕ (**greedy**), prompt-selection rule f_P (natural prefixes), and the score that composes them. We plot the rate of verbatim reproduction of the suffix given the prefix, with 95% Clopper–Pearson confidence intervals (See Appendix). Each non-member rate is a false-positive floor: the reproduction rate attributable to predictability rather than memorization. For 10-token suffixes it is 1.82% against 7.54% on members, so up to $\sim 24\%$ of what this greedy test would flag as extraction sits at or below the floor. For 50-token suffixes, the false-positive floor is close to zero.

deterministic, the test’s score is binary: discoverable extraction prompts with the natural prefix, decodes by taking the argmax (top-1) token at each step, and scores 1 if the generated continuation exactly matches the candidate and 0 otherwise (Section 2). With a binary score the threshold is degenerate because any cutoff in $(0, 1)$ yields the same test: “the greedy completion equals the candidate.” There is nothing to calibrate, as this only exposes a single operating point.

Figure 3 contrasts greedy verbatim generation on training members with the matched non-member controls. We deliberately call this a comparison of “verbatim generation” rather than “verbatim extraction” because the non-member controls, by definition, were never in training and so cannot be memorized. Any verbatim generation of a control sequence is therefore a false positive, and its rate across the control sample is the false-positive rate. We also refer to this rate as the **false-positive floor**: the level of verbatim generation reached where memorization is impossible.

For 10-token suffixes, the non-member floor is large: 1.82%, compared to a 7.54% generation rate on training data. If one ran the experiment only on training data and claimed extraction, the observed result would be the 7.54% “extraction” rate. But roughly one quarter ($1.82/7.54 \approx 24\%$) of that apparent training-data “extraction” rate is matched by the rate at which the same test flags short non-member continuations. Because the 5,000 member sequences are sampled, the same ambiguity carries over to the population rate that this sample estimates. The conformal comparison does not show that any particular generated member suffix is not memorized; rather, it shows that a verbatim match at this suffix length is not a sufficiently specific membership signal because, under the same measurement procedure, short non-member continuations cross the threshold at a nontrivial rate. Therefore, an experiment that omits this comparison risks attributing the entire 7.54% member generation rate to memorization, even though at most $\approx 24\%$ of it could be false positives that predictability alone would produce.¹¹

The picture is very different for 50-token suffixes. The verbatim training-data generation rate is much smaller (0.74%), but the non-member floor is close to zero (0.02%). At this length, the false-positive floor

¹¹At most $\approx 24\%$ of the matches are false positives. There are fewer if some of these members are truly memorized. Because the greedy-decoding score is binary, we cannot tell which positives are the false ones: every match between a greedy-generated continuation and a training suffix looks the same (i.e., scores 1), whether it comes from memorization or from general predictability. The probabilistic test below (Figure 4) can do better: its continuous score singles out the suspected false positives—the members whose scores overlap the controls’.

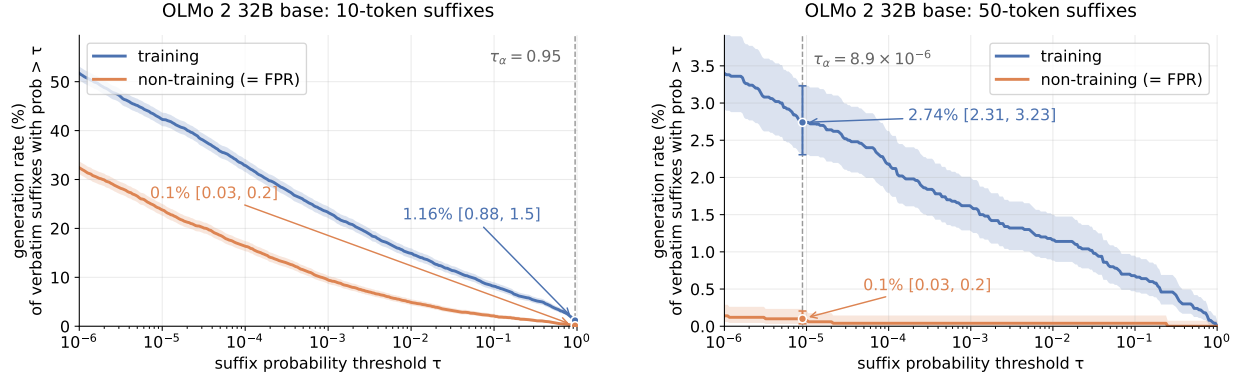


Figure 4: **Generation-rate complementary CDFs over the threshold τ for OLMo 2 32B base.** We plot the fraction of 10-token (left) and 50-token (right) verbatim suffixes generated with probability above threshold τ , for training members (Wikipedia sequences from January 1, 2021 through the December 2023 cutoff) and matched non-member controls (Wikipedia sequences from January 1, 2024–December 31, 2024). The control curve is the FPR at each threshold. The calibrated memorization rate is the difference between the training generation rate and the calibrated control FPR. As in the greedy comparison, the two sequences responsible for the 50-token control curve’s non-zero FPR at $\alpha = 0.1\%$ are contaminants, so the calibrated memorization rate is a conservative underestimate. See Appendix for further details on confidence intervals.

is negligible: matched non-members almost never cross the threshold, so the observed member generations are not plausibly explained by predictability seen in the shorter 10-token setting. In fact, the two sequences responsible for the non-zero 50-token false-positive floor are contaminants in the control pool (Section 3.2), which we discuss further below (Figure 5).

Verbatim probabilistic extraction. In contrast to greedy discoverable extraction, probabilistic extraction scores each candidate by a continuous quantity: the generation probability $q_{\theta,\phi}(\mathbf{s} \mid \mathbf{p}) \in [0, 1]$ the model assigns to the suffix $\mathbf{s}$ given its prefix $\mathbf{p}$ (computed here under top- k decoding with $k = 40$, as in Hayes et al. [27] and Cooper et al. [19]). Because the score is graded rather than binary, we can sweep the threshold across many operating points, rather than the single one greedy extraction exposes. Further, because a greedy match is the top-1 special case of top-40 reachability, the greedy-extractable suffixes are a subset of the probabilistically-extractable ones. The probabilistic version of the test therefore certifies a superset: a broader (still FPR-controlled) notion of memorization.¹² For the same model, data, and suffix-length settings as in Figure 3, Figure 4 plots the complementary CDF of these scores: a point (τ, y) means $y\%$ of sequences score above τ (i.e., $q_{\theta,\phi}(\mathbf{s} \mid \mathbf{p}) > \tau$).

Following the conformal test (Section 3.1), the control scores calibrate this threshold. For a target false-positive rate α , we deem a candidate memorized when its conformal p -value $\hat{p}$ (Equation 6) is at most α —equivalently, when its score exceeds τ_α , the $(1 - \alpha)$ quantile of the control scores (i.e., $\hat{p} \leq \alpha \iff q_{\theta,\phi}(\mathbf{s} \mid \mathbf{p}) > \tau_\alpha$). By construction, at most an α fraction of non-members exceed τ_α , so this holds the false-positive rate at or below α . The conformal test therefore formalizes the distinction between chance generation and extraction. A generation probability, on its own, is not evidence of memorization; only once it clears τ_α do we call it an extraction probability, since exceeding the calibrated threshold is what licenses the memorization claim.

The two panels of Figure 4 show the same effect as the greedy discoverable extraction results, but in finer detail across the full range of thresholds: at short suffix lengths, the model assigns nontrivial probability to generating non-member sequences. At 10 tokens, many continuations are highly predictable, so non-members carry high scores. Holding the false-positive rate to 0.1% therefore forces τ_α up to ≈ 0.95 : the test flags a candidate as memorized only when the model generates it with near-certainty. At 50 tokens, the control curve instead collapses to near zero across almost the entire range of thresholds. Non-members are essentially never generated verbatim, so the threshold τ_α that gives a 0.1% FPR is itself tiny—orders of magnitude below the 10-token threshold—and the member curve sits far above it.

¹²The near-verbatim conformal test (Section 6) is a third, still-broader instance of this reachability hierarchy.

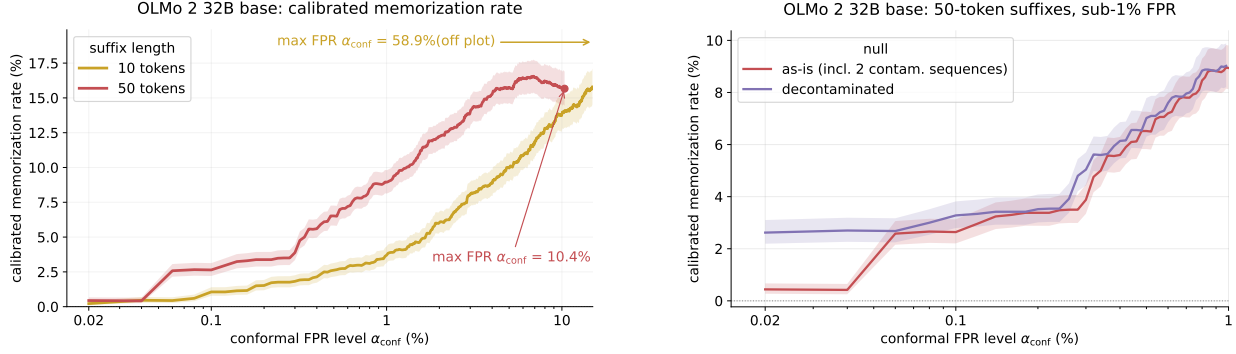


Figure 5: **Calibrated memorization rate for OLMo 2 32B base on Wikipedia.** Across finite-sample false-positive levels α_{conf} , we plot the empirical calibrated memorization rate $\hat{M} = \hat{G} - \alpha_{\text{conf}}$ (**left**): the fraction of training suffixes generated above the conformal threshold $\tau_\alpha(\hat{G})$, minus the finite-sample false-positive level $\alpha_{\text{conf}} = \lfloor \alpha(n+1) \rfloor / (n+1)$ provided by that threshold (Equation 10). We plot rates for both 10-token and 50-token suffixes, using the same 50-token prefixes. We zoom in on the 50-token sub-1% regime (**right**), comparing the calibrated memorization rate before and after removing training-sequence contamination from the control sample (2 verified contaminants removed). The contaminants sit at the top of the control distribution, depressing the apparent calibrated memorization rate. See Appendix for details on confidence intervals.

4.2 Computing calibrated memorization rates

Each operating point in Section 4.1 has two associated rates: the rate at which member suffixes are generated above τ_α , and the rate at which matched non-member suffixes also clear the same threshold. The member rate is therefore not, by itself, a clean memorization rate. At any fixed threshold, some non-members also clear the threshold; those are false positives, and the non-member rate estimates their frequency. We alluded to this same point when discussing verbatim greedy discoverable extraction (Figure 3). In that setting, where there is only one operating point, we discussed how the greedy-generated false-positive floor is the part of the member generation rate that is not uniquely indicative of membership (and therefore not uniquely attributable to memorization). For probabilistic extraction, we make the same calibration at each operating point, defining a calibrated memorization rate.

Definition 4.1 (Calibrated memorization rate). Fix an extraction procedure (Definition 2.2) and let $\text{score}(\mathbf{s})$ be a candidate sequence $\mathbf{s}$ ’s extraction probability under that procedure. For a threshold τ , write $G(\tau) = \Pr[\text{score}(\mathbf{s}) > \tau \mid \text{membership}_\varepsilon(\mathbf{s}) = 1]$ for the population member generation rate, and $\text{FPR}(\tau) = \Pr[\text{score}(\mathbf{s}) > \tau \mid \text{membership}_\varepsilon(\mathbf{s}) = 0]$ for the population false-positive rate on matched non-members. The **calibrated memorization rate** at threshold τ is

$$M(\tau) := G(\tau) - \text{FPR}(\tau), \quad (9)$$

measuring how much the above-threshold generation rate for members exceeds the above-threshold generation rate for matched non-members under the fixed extraction procedure.

Under the matched-control assumption that matched non-members represent non-memorized members for this score, $M(\tau)$ is a conservative lower bound on the total latent fraction of members that are memorized.¹³ It is only a lower bound because not all memorization surfaces in the extraction-probability signal we measure: a suffix can be memorized yet score below τ under the given extraction procedure, in which case no memorization claim is made. And so, $M(\tau)$ is an extractability-based lower bound on total latent memorization, not an estimate of it (Section 3.1).

¹³Let π be the fraction of members that are memorized, and let r_τ be the rate at which memorized members clear τ (the extraction-signal-based true-positive rate; $r_\tau \leq 1$, since a memorized member need not score above τ). By the matched-control assumption, non-memorized members clear τ at the same rate as matched non-members, $\text{FPR}(\tau)$, so $G(\tau) = \pi r_\tau + (1 - \pi) \text{FPR}(\tau)$ and $M(\tau) = G(\tau) - \text{FPR}(\tau) = \pi(r_\tau - \text{FPR}(\tau)) \leq \pi$. The bound holds pointwise in τ , so we fix τ in advance, or set it by a target FPR, rather than choosing it post hoc to maximize $M(\tau)$.

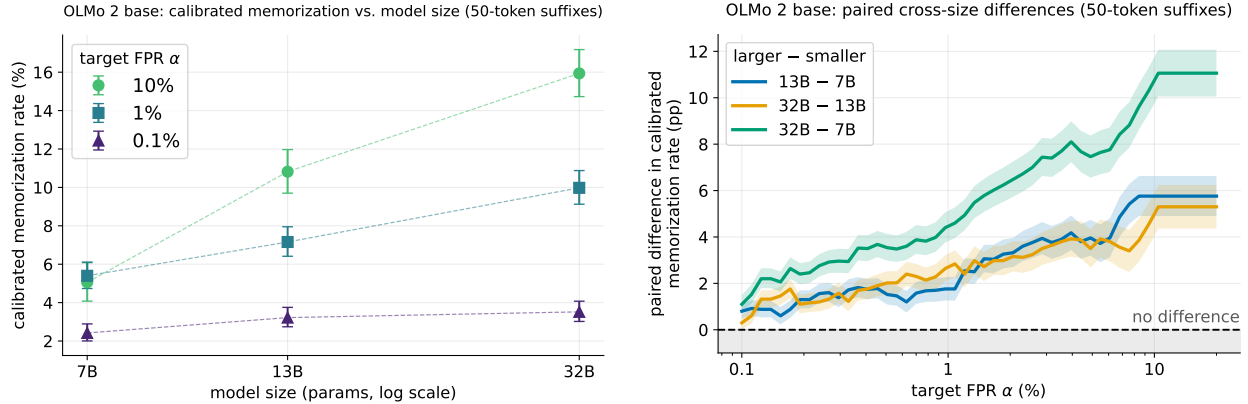


Figure 6: **Calibrated verbatim memorization versus model size for OLMo 2 base.** Using the same member sequences and matched non-member controls (50-token prefixes and suffixes) for three model sizes, we show (**left**) the absolute calibrated memorization rate $\hat{M}$ (Equation 10) at three target FPRs $\alpha \in \{0.1\%, 1\%, 10\%\}$. This plot answers, “what is each model’s absolute calibrated rate?”; the confidence intervals are computed separately for each model’s rate, so visually comparing their overlap is not the right test for judging a size effect. Instead, for a size effect, the relevant question is paired: “on the same member sequences, how often does one model generate a given suffix above its own calibrated threshold when the other does not?” We plot these paired cross-size differences swept across false-positive levels, with pointwise Newcombe paired-proportion intervals [44] (**right**). Intervals entirely above the no-difference line support the inferential claim that the larger model has a higher calibrated memorization rate at that operating point. See Appendix for more details on confidence intervals.

Empirical estimation. Definition 4.1 is stated in terms of population rates. In practice, we estimate the calibrated memorization rate at a specific conformal operating point: the finite-sample false-positive level $\alpha_{\text{conf}} = \lfloor \alpha(n+1) \rfloor / (n+1)$ that the conformal threshold τ_α supplies. That is, to estimate M , we compute

$$\hat{M} := \hat{G} - \alpha_{\text{conf}}, \quad (10)$$

the empirical member generation rate $\hat{G}$ above the conformal threshold τ_α minus the finite-sample false-positive level α_{conf} that threshold yields.

4.3 Model scale, sample size, and inferential claims

We now apply the calibrated memorization rate across model scale, comparing the OLMo-2 base models (7B, 13B, and 32B parameters) on the same 50-token Wikipedia candidates.

5 Census experiments

For extractable memorization, the question of interest isn’t always about population rates. In other cases, we are interested in understanding the extent to which a single specific training document is memorized. For this type of question, we instead want to take a **census** approach and evaluate the whole document. Here, there is no population we are sampling from, so the conformal test doesn’t apply; there is no notion of an empirical FPR (that estimates a population FPR). But we can still perform a matched comparison with non-training data. Here, we do exactly this for books and Llama 3.1 models. We use the sliding-window extraction procedure from Cooper et al. [19] with verbatim probabilistic extraction. We run 4 pairs of books, where each pair matches an author’s book in Books3 (in Llama 3.1’s training data) with a more recent book that post-dates Llama 3.1’s training cutoff. This is a best effort at running a matched comparison: within each pair, the books are by the same author, in the same genre, and (in two cases) even in the same series, with overlapping characters and plot elements. By calibrating against the non-training book in each pair, we set a book-specific threshold $\tau^{(\text{book})}$ based on the highest-scoring sequence from the non-training book.

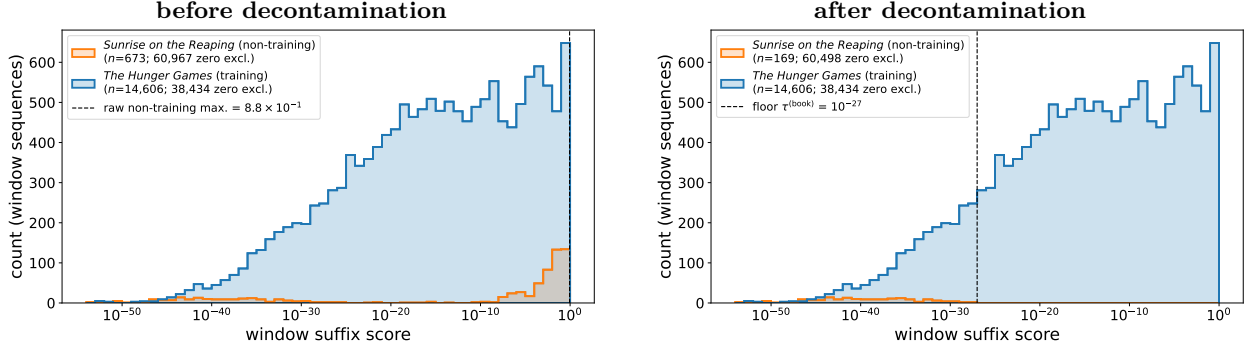


Figure 7: **Decontaminating the control pool to identify the floor.** We remove sequences from the non-training book (*Sunrise on the Reaping*) that are actually members of the training data, and set the threshold $\tau^{(\text{book})}$ based on the highest-scoring sequence of unique text from the non-training book.

5.1 Floor estimation and decontaminating nulls

To select each pair’s threshold, we examine the sequence probabilities for both the in-training and non-training book. We show this here in Figure 7 for the pair of books by Suzanne Collins. Even though the chosen non-training book, as a document, is not included in training, that book may quote material from other sources that is in the training data. Such sequences contaminate the control pool; they are actually members, not non-members.

We develop a procedure for cleaning the control pool to remove those sequences. Whereas in the past we did this work manually, here we developed an agent skill with web search to assist the process. In the case of *Sunrise on the Reaping* (the non-training Suzanne Collins book), the high-scoring sequences come from Edgar Allen Poe’s “The Raven,” which is quoted repeatedly throughout the book, as well as verbatim quotes from *The Hunger Games* [14], which is also in the training data. We cull those sequences from the control pool, and using the highest-scoring sequence of unique text to the non-training book to set the floor $\tau^{(\text{book})}$. Note that the probability of the floor-setting sequence is so small that the mantissa is unreliable, with respect to floating point precision. To be conservative, we discard the mantissa and round up the floor to the next decade. For verbatim probabilistic extraction and this pair, we set $\tau^{(\text{book})} = 10^{-27}$.

We can then plot where memorization occurs in *The Hunger Games* with respect this floor, following the visualization procedure in Cooper et al. [19] for *The Hunger Games* in Figure 8.

5.2 The strength of memorization evidence varies

The calibrated thresholds we obtain are extremely small; as noted above, they are so small that the mantissa reflects floating-point noise. For sequences that just clear this floor, we can think of them as more weakly memorized than those that are far above it. The heatmap in Figure 8 already conveys this: darker regions indicate stronger suffix probabilities, indicating stronger evidence for memorization. It can also be useful to convey this information quantitatively. We do this with the **margin** above the threshold that a sequence obtains: the number of orders of magnitude of the sequence’s probability has above the floor $\tau^{(\text{book})}$. For a candidate sequence $\mathbf{s}$, we compute

$$\Delta(\mathbf{s}) = \log_{10} \frac{\text{score}(\mathbf{s})}{\tau^{(\text{book})}}, \quad (11)$$

the number of orders of magnitude its score exceeds the floor ($\Delta(\mathbf{s}) > 0$ means above the floor). Figure 9 reads off robustness this way: it reports coverage—the percentage of the book that can be extracted, with respect to this specific extraction procedure using 50-token prefixes and suffixes [19])—as the threshold is pushed up from $\tau^{(\text{book})}$. As we increase the threshold, coverage of course decreases, but the sequences that reflect that threshold survive a higher level of stringency.

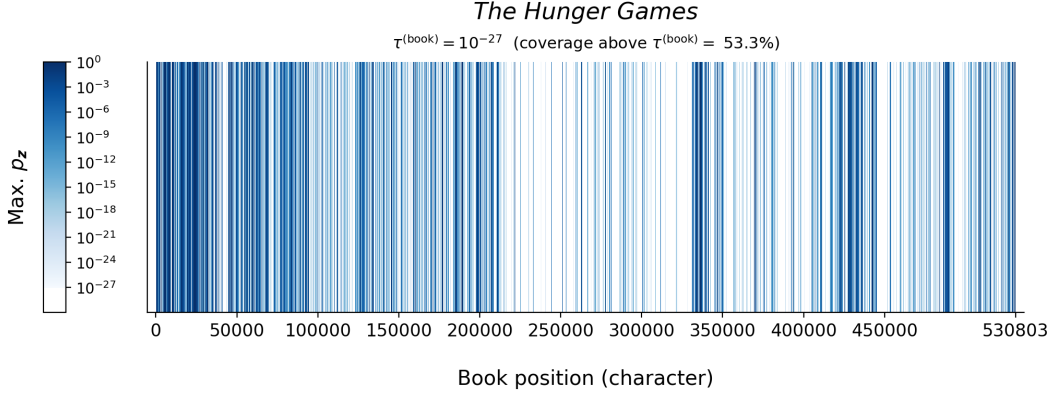


Figure 8: **Visualizing memorization with a heatmap.** We show how memorization varies, with respect to extraction probability as the signal, for *The Hunger Games* and the floor calibrated with a matched comparison using *Sunrise on the Reaping*.

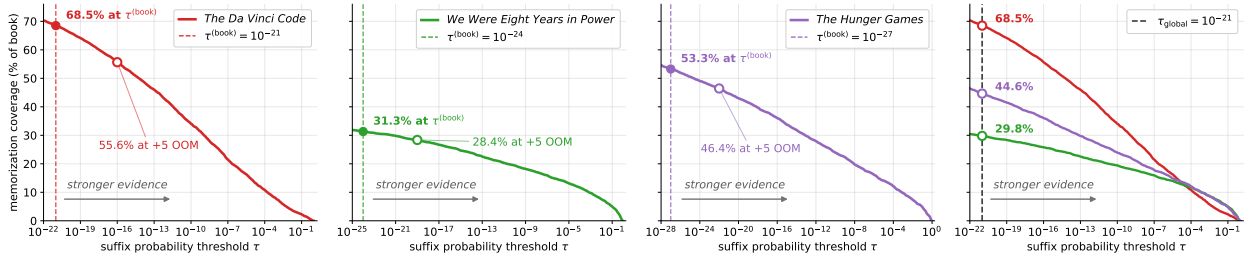


Figure 9: Showing how coverage varies as we demand stronger evidence for memorization (i.e., by increasing the suffix probability threshold under consideration).

6 A definition for extractable memorization: realistic extractability vs. memorization

Candidate sequences that clear the threshold are deemed memorized, with respect to this matched comparison using the fixed extraction procedure. But the thresholds we identify for distinguishing memorization are often very low, especially for the books census experiments. Even though clearing this threshold licenses calling the generation probability an extraction probability—as the literature thus far has discussed extraction probabilities—sequences even orders of magnitude larger than the threshold can have such low “extraction” probabilities that, realistically, they would never actually be extracted in practice in outputs via sampling. It therefore seems like a bit of a misnomer to call this “extractable memorization.”

We therefore suggest a revised definition for **extractable memorization**.

Definition 6.1 (Extractable memorization). Fix an extraction procedure (Definition 2.2) and a query budget n . A candidate s is **extractably memorized** if both hold:

- (i) *it is memorized*: its score, $\text{score}(s)$, clears the matched-comparison threshold, providing calibrated evidence of memorization (Section 3.1); and
- (ii) *it is reproducible within a realistic query budget*: it can be produced reliably in outputs within query budget n , whether by sampling (so that $1 - (1 - \text{score}(s))^n$ is near 1) or by another decoding procedure of comparable cost.

We give a sense for this distinction in Figure 10, where, as a function of attempt budget n , we plot results for both verbatim probabilistic extraction (which is exact, via teacher-forcing [27, 19]) and near-verbatim probabilistic extraction (which in theory is a superset of verbatim, but in practice we compute this with

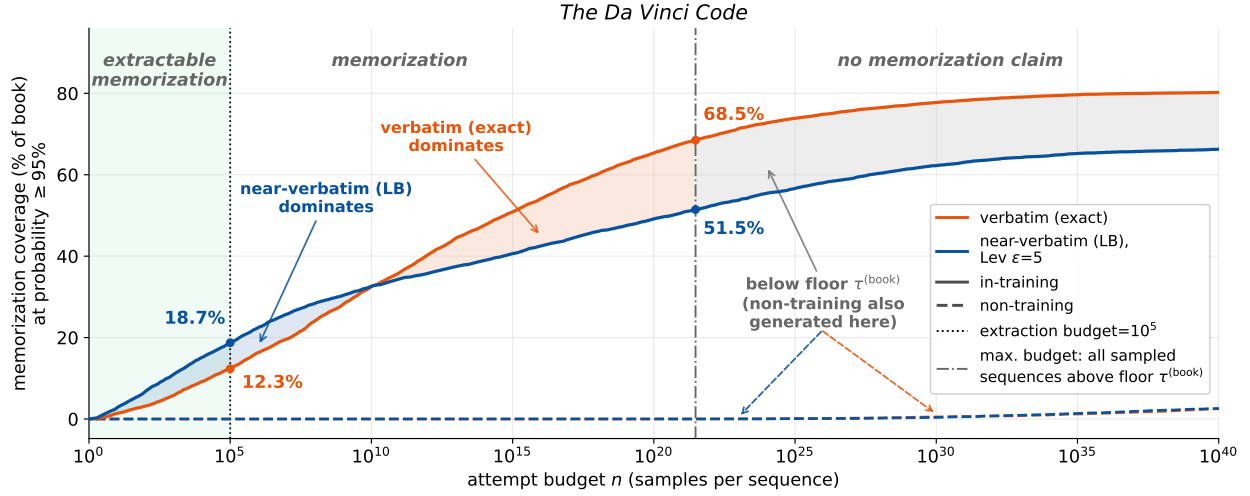


Figure 10: We take an attempt budget view for extracting memorized sequences, following Hayes et al. [27]. We show how verbatim (exact) probabilistic extraction and the near-verbatim (lower bound [20]) probabilistic extraction vary over the budget. As a reference, we also plot the threshold and curves for non-training data in both settings; coverage in those settings exceeds 0 only after passing the maximum budget associated with $\tau^{(\text{book})}$.

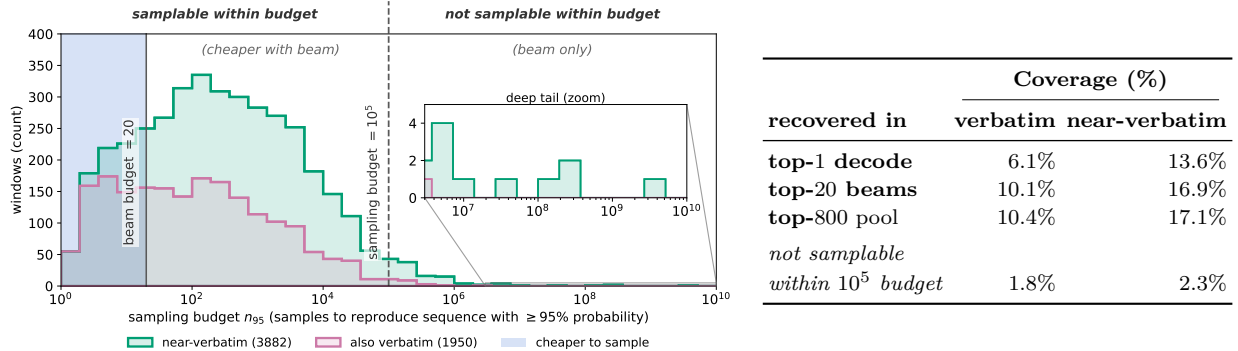


Figure 11: Showing which sequences the deterministic, near-verbatim probabilistic extraction lower-bound algorithm [20] surfaces within a budget comparable to ≈ 20 independent samples.

the efficient lower-bound algorithm from Cooper et al. [20]). This is why the near-verbatim coverage values start out dominating the verbatim ones, but at a relatively low probability the exact verbatim computation captures more mass (that the efficient lower-bound algorithm misses). For illustration purposes, we pick 10^5 samples as the realistic query budget, and highlight extraction that can be sampled in outputs within that budget, which is significantly less than the total amount of memorization that the matched comparison reveals (the budget line associated with the threshold $\tau^{(\text{book})}$).

While Figure 10 shows how much extraction is possible with *random sampling*, this is of course not the only way to decode outputs. There may exist other decoding algorithms that can obtain lower-probability (but still memorized) sequences in outputs. To give a sense for this, in Figure 11 we show what Cooper et al.’s beam-search-based near-verbatim lower-bound algorithm returns deterministically, with cost comparable to 20 samples. The algorithm returns a set of sequences (an 800-continuation pool for each candidate), which we check for extraction.

7 Takeaways

It’s time to retire the “monkey at the typewriter” argument against valid extraction.¹⁴

TODO:

Acknowledgments

We thank Katherine Lee for useful discussions and feedback on earlier versions of this draft.

References

- [1] Ahmed Ahmed, A. Feder Cooper, Sanmi Koyejo, and Percy Liang. Extracting books from production language models. *arXiv preprint arXiv:2601.02671*, 2026.
- [2] Federico Barbero, Xiangming Gu, Christopher A. Choquette-Choo, Chawin Sitawarin, Matthew Jagielski, Itay Yona, Petar Veličković, Ilia Shumailov, and Jamie Hayes. Extracting alignment data in open models, 2025. URL <https://arxiv.org/abs/2510.18554>.
- [3] Dan Brown. *The Da Vinci Code*. Transworld Publishers; Bantam Press; Corgi, 2003.
- [4] Dan Brown. *The Secret of Secrets*. Doubleday; Penguin Random House LLC, 2025.
- [5] Hannah Brown, Katherine Lee, Fatemehsadat Miresghallah, Reza Shokri, and Florian Tramèr. What Does it Mean for a Language Model to Preserve Privacy? In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, FAccT ’22, pp. 2280–2292, New York, NY, USA, 2022. Association for Computing Machinery. URL <https://doi.org/10.1145/3531146.3534642>.
- [6] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX security symposium (USENIX security 19)*, pp. 267–284, 2019.
- [7] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, et al. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pp. 2633–2650, 2021.
- [8] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. Membership inference attacks from first principles. In *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 1897–1914. IEEE, 2022.
- [9] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. Quantifying Memorization Across Neural Language Models. In *International Conference on Learning Representations*, 2023.
- [10] Kent K. Chang, Mackenzie Cramer, Sandeep Soni, and David Bamman. Speak, Memory: An Archaeology of Books Known to ChatGPT/GPT-4. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7312–7327, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.453. URL <https://aclanthology.org/2023.emnlp-main.453/>.
- [11] Alexandra Chouldechova, A. Feder Cooper, Solon Barocas, Abhinav Palia, Dan Vann, and Hanna Wallach. Comparison requires valid measurement: Rethinking attack success rate comparisons in AI red teaming. In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems Position Paper Track*, 2025. URL <https://openreview.net/forum?id=d7hqAhLvWG>.
- [12] Ta-Nehisi Coates. *We Were Eight Years in Power*. One World; Random House; Penguin Random House LLC, 2017.

¹⁴He was very early at OpenAI, is very tired, and has chronic carpal tunnel. He’ll be retiring to Lake Como.

-
- [13] Ta-Nehisi Coates. *The Message*. One World; Random House; Penguin Random House LLC, 2024.
- [14] Suzanne Collins. *The Hunger Games*. Scholastic Children’s Books; Scholastic Ltd, 2008.
- [15] Suzanne Collins. *Sunrise on the Reaping*. Scholastic Press; Scholastic, 2025.
- [16] A. Feder Cooper and James Grimmelman. The Files are in the Computer: Copyright, Memorization, and Generative AI. *arXiv preprint arXiv:2404.12590*, 2024.
- [17] A. Feder Cooper, Katherine Lee, James Grimmelman, Daphne Ippolito, Christopher Callison-Burch, Christopher A. Choquette-Choo, Niloofar Mireshghallah, Miles Brundage, David Mimno, Madiha Zahrah Choksi, Jack M. Balkin, Nicholas Carlini, Christopher De Sa, Jonathan Frankle, Deep Ganguli, Bryant Gipson, Andres Guadamuz, Swee Leng Harris, Abigail Z. Jacobs, Elizabeth Joh, Gautam Kamath, Mark Lemley, Cass Matthews, Christine McLeavey, Corynne McSherry, Milad Nasr, Paul Ohm, Adam Roberts, Tom Rubin, Pamela Samuelson, Ludwig Schubert, Kristen Vaccaro, Luis Villa, Felix Wu, and Elana Zeide. Report of the 1st Workshop on Generative AI and Law. *arXiv preprint arXiv:2311.06477*, 2023.
- [18] A. Feder Cooper, Christopher A Choquette-Choo, Miranda Bogen, Matthew Jagielski, Katja Filippova, Ken Ziyu Liu, Alexandra Chouldechova, Yangsibo Huang Jamie Hayes, et al. Machine unlearning doesn’t do what you think: Lessons for generative ai policy, research, and practice. *arXiv preprint arXiv:2412.06966*, 2024.
- [19] A. Feder Cooper, Aaron Gokaslan, Amy B. Cyphert, Christopher De Sa, Mark A. Lemley, Daniel E. Ho, and Percy Liang. Extracting memorized pieces of (copyrighted) books from open-weight language models. *arXiv preprint arXiv:2505.12546*, 2025.
- [20] A. Feder Cooper, Mark A. Lemley, Christopher De Sa, Lea Duesterwald, Allison Casasola, Jamie Hayes, Katherine Lee, Daniel E. Ho, and Percy Liang. Estimating near-verbatim extraction risk in language models with decoding-constrained beam search. *arXiv preprint arXiv:2603.24917*, 2026.
- [21] Vitaly Feldman. Does learning require memorization? a short tale about a long tail. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2020, pp. 954–959, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450369794.
- [22] Joseph Finder. *Killer Instinct*. Orion Publishing Group, 2006.
- [23] Joseph Finder. *The Oligarch’s Daughter*. Harper; HarperCollins, 2025.
- [24] Gemini Team, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [25] Aaron Grattafiori et al. The Llama 3 Herd of Models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [26] Jamie Hayes, Ilia Shumailov, Christopher A. Choquette-Choo, Matthew Jagielski, Georgios Kaissis, Milad Nasr, Meenatchi Sundaram Muthu Selva Annamalai, Niloofar Mireshghallah, Igor Shilov, Matthieu Meeus, Yves-Alexandre de Montjoye, Katherine Lee, Franziska Boenisch, Adam Dziedzic, and A. Feder Cooper. Exploring the limits of strong membership inference attacks on large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=x0i7wvRLHK>.
- [27] Jamie Hayes, Marika Swanberg, Harsh Chaudhari, Itay Yona, Ilia Shumailov, Milad Nasr, Christopher A. Choquette-Choo, Katherine Lee, and A. Feder Cooper. Measuring memorization in language models via probabilistic extraction. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 9266–9291, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. URL <https://aclanthology.org/2025.naacl-long.469/>.

-
- [28] John Hughes, Sara Price, Aengus Lynch, Rylan Schaeffer, Fazl Barez, Sanmi Koyejo, Henry Sleight, Erik Jones, Ethan Perez, and Mrinank Sharma. Best-of-N Jailbreaking, 2024. URL <https://arxiv.org/abs/2412.03556>.
- [29] Daphne Ippolito, Florian Tramèr, Milad Nasr, Chiyuan Zhang, Matthew Jagielski, Katherine Lee, Christopher Choquette Choo, and Nicholas Carlini. Preventing Generation of Verbatim Memorization in Language Models Gives a False Sense of Privacy. In C. Maria Keet, Hung-Yi Lee, and Sina Zarrieß (eds.), *Proceedings of the 16th International Natural Language Generation Conference*, pp. 28–53, Prague, Czechia, September 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.inlg-main.3. URL <https://aclanthology.org/2023.inlg-main.3/>.
- [30] Najeeb Jebreel, Mona Khalil, David Sánchez, and Josep Domingo-Ferrer. Revisiting the LiRA Membership Inference Attack Under Realistic Assumptions, 2026. URL <https://arxiv.org/abs/2603.07567>.
- [31] Antonia Karamolegkou, Jiaang Li, Li Zhou, and Anders Søgaard. Copyright Violations and Large Language Models, 2023. URL <https://arxiv.org/abs/2310.13771>.
- [32] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating Training Data Makes Language Models Better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, volume 1, pp. 8424–8445, 2022.
- [33] Katherine Lee, A. Feder Cooper, James Grimmelmänn, and Daphne Ippolito. AI and Law: The Next Generation. *SSRN*, 2023. <http://dx.doi.org/10.2139/ssrn.4580739>.
- [34] Katherine Lee, A. Feder Cooper, and James Grimmelmänn. Talkin’ ’Bout AI Generation: Copyright and the Generative-AI Supply Chain. *arXiv preprint arXiv:2309.08133*, 2023.
- [35] Ken Ziyu Liu, Christopher A. Choquette-Choo, Matthew Jagielski, Peter Kairouz, Sanmi Koyejo, Percy Liang, and Nicolas Papernot. Language models may verbatim complete text they were not explicitly trained on, 2025. URL <https://arxiv.org/abs/2503.17514>.
- [36] Xinyue Liu, Niloofar Mireshghallah, Jane C. Ginsburg, and Tuhin Chakrabarty. Alignment Whack-a-Mole : Finetuning Activates Verbatim Recall of Copyrighted Books in Large Language Models, 2026.
- [37] Bruce T. Lowerre. *The HARPY Speech Recognition System*. PhD thesis, Carnegie Mellon University, Pittsburgh, PA, 1976. PhD Dissertation.
- [38] Pratyush Maini, Hengrui Jia, Nicolas Papernot, and Adam Dziedzic. LLM Dataset Inference: Did you train on my dataset?, 2024. URL <https://arxiv.org/abs/2406.06443>.
- [39] John X. Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. How much do language models memorize?, 2025. URL <https://arxiv.org/abs/2505.24832>.
- [40] Randall Munroe. Predictive Models (xkcd #2169). <https://xkcd.com/2169/>, June 2019. Accessed: 2026-07-13.
- [41] Milad Nasr, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Eric Wallace, Florian Tramèr, and Katherine Lee. Scalable Extraction of Training Data from (Production) Language Models. *arXiv preprint arXiv:2311.17035*, 2023.
- [42] Milad Nasr, Javier Rando, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Florian Tramèr, and Katherine Lee. Scalable Extraction of Training Data from Aligned, Production Language Models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=vjel3nWP2a>.
- [43] Gonzalo Navarro. A guided tour to approximate string matching. *ACM Comput. Surv.*, 33(1):31–88, March 2001. ISSN 0360-0300. doi: 10.1145/375360.375365. URL <https://doi.org/10.1145/375360.375365>.

-
- [44] Robert G. Newcombe. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in Medicine*, 17(22):2635–2650, 1998.
- [45] Henrik Nolte, Michèle Finck, and Kristof Meding. Machine learners should acknowledge the legal implications of large language models as personal data, 2025. URL <https://arxiv.org/abs/2503.01630>.
- [46] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Allyson Ettinger, Michal Guerquin, David Heineman, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Jake Poznanski, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious, 2025. URL <https://arxiv.org/abs/2501.00656>.
- [47] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ACL ’02, pp. 311–318, USA, 2002. Association for Computational Linguistics. doi: 10.3115/1073083.1073135. URL <https://doi.org/10.3115/1073083.1073135>.
- [48] Avi Schwarzschild, Zhili Feng, Pratyush Maini, Zachary C Lipton, and J Zico Kolter. Rethinking llm memorization through the lens of adversarial compression. *arXiv preprint arXiv:2404.15146*, 2024.
- [49] Glenn Shafer and Vladimir Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9, 2008.
- [50] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, pp. 3–18. IEEE, 2017.
- [51] Gemma Team et al. Gemma 2: Improving Open Language Models at a Practical Size, 2024. URL <https://arxiv.org/abs/2408.00118>.
- [52] Trishita Tiwari, Ari Trachtenberg, and G. Edward Suh. Prior Aware Memorization: An Efficient Metric for Distinguishing Memorization from Generalization in Large Language Models, 2026. URL <https://arxiv.org/abs/2602.18733>.
- [53] Johnny Tian-Zheng Wei, Maggie Wang, Ameya Godbole, Jonathan H. Choi, and Robin Jia. Interrogating llm design under a fair learning doctrine, 2025. URL <https://arxiv.org/abs/2502.16290>.
- [54] Chiyuan Zhang, Daphne Ippolito, Katherine Lee, Matthew Jagielski, Florian Tramèr, and Nicholas Carlini. Counterfactual Memorization in Neural Language Models, 2023. URL <https://arxiv.org/abs/2112.12938>.
- [55] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models, 2023. URL <https://arxiv.org/abs/2307.15043>.

A Population sample experiments

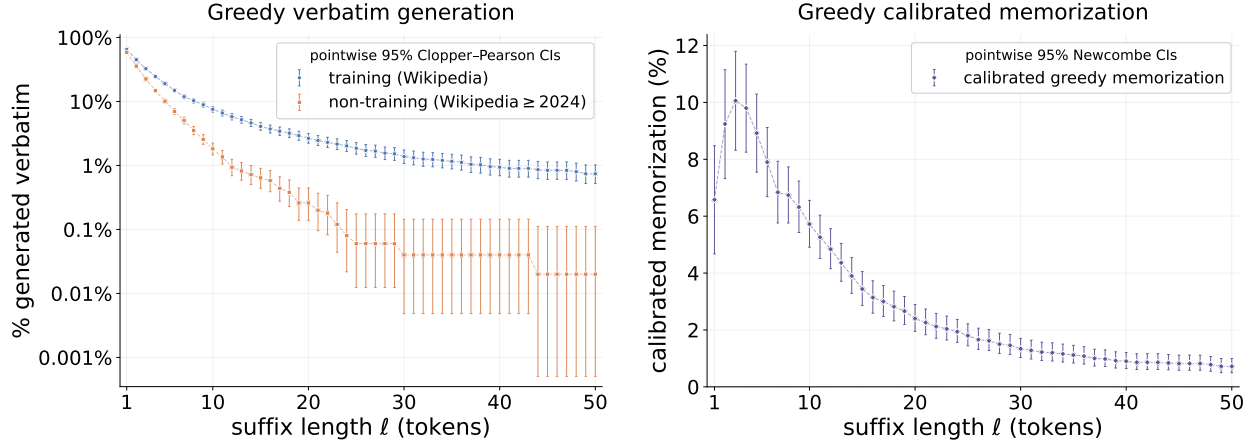


Figure 12: Greedy discoverable extraction as a function of suffix lengths (OLMo 2 32B on Wikipedia).

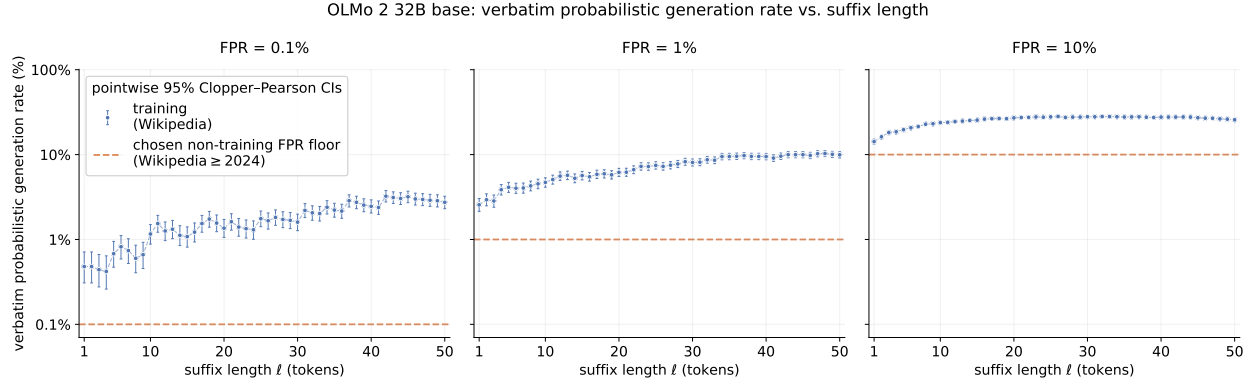


Figure 13: Probabilistic generation rate as a function of suffix length (OLMo 2 32B on Wikipedia). We fix the FPR (verbatim generation rate of non-members) and plot the rate for members.

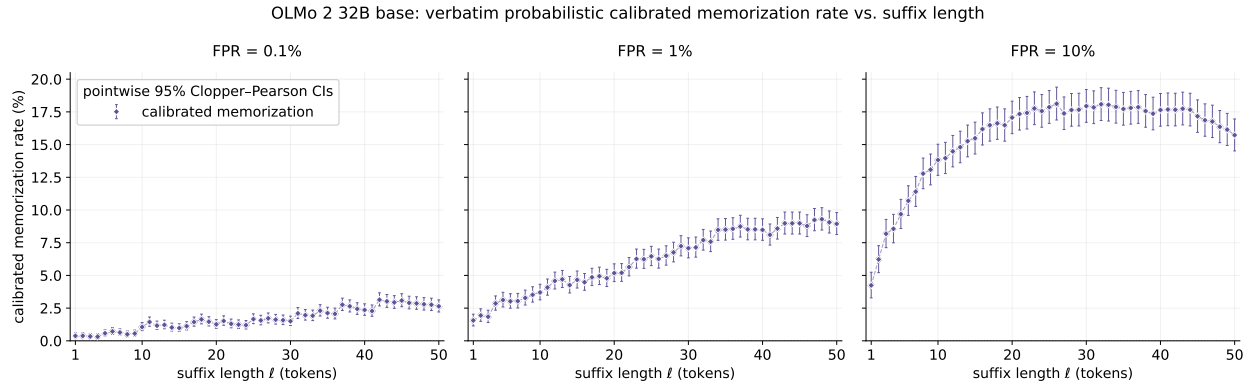
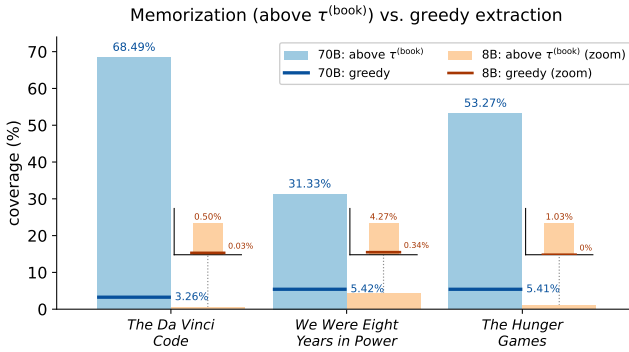


Figure 14: For verbatim probabilistic extraction, the calibrated memorization rate as a function of suffix length at different fixed FPR.

B Census experiments

Table 1: We run the single-document census experiments with Llama 3.1 base models (8B and 70B), whose pretraining data cutoff is December 2023. For each author we pair an in-training book present in the Books3 corpus (published before the cutoff) with a matched control book by the same author published after the cutoff.

Author	In-training (Books3)	Post-cutoff (control)
Dan Brown	<i>The Da Vinci Code</i> [3] (March 2003)	<i>The Secret of Secrets</i> [4] (September 2025)
Suzanne Collins	<i>The Hunger Games</i> [14] (September 2008)	<i>Sunrise on the Reaping</i> [15] (March 2025)
Ta-Nehisi Coates	<i>We Were Eight Years in Power</i> [12] (October 2017)	<i>The Message</i> [13] (October 2024)
Joseph Finder	<i>Killer Instinct</i> [22] (May 2006)	<i>The Oligarch’s Daughter</i> [23] (January 2025)

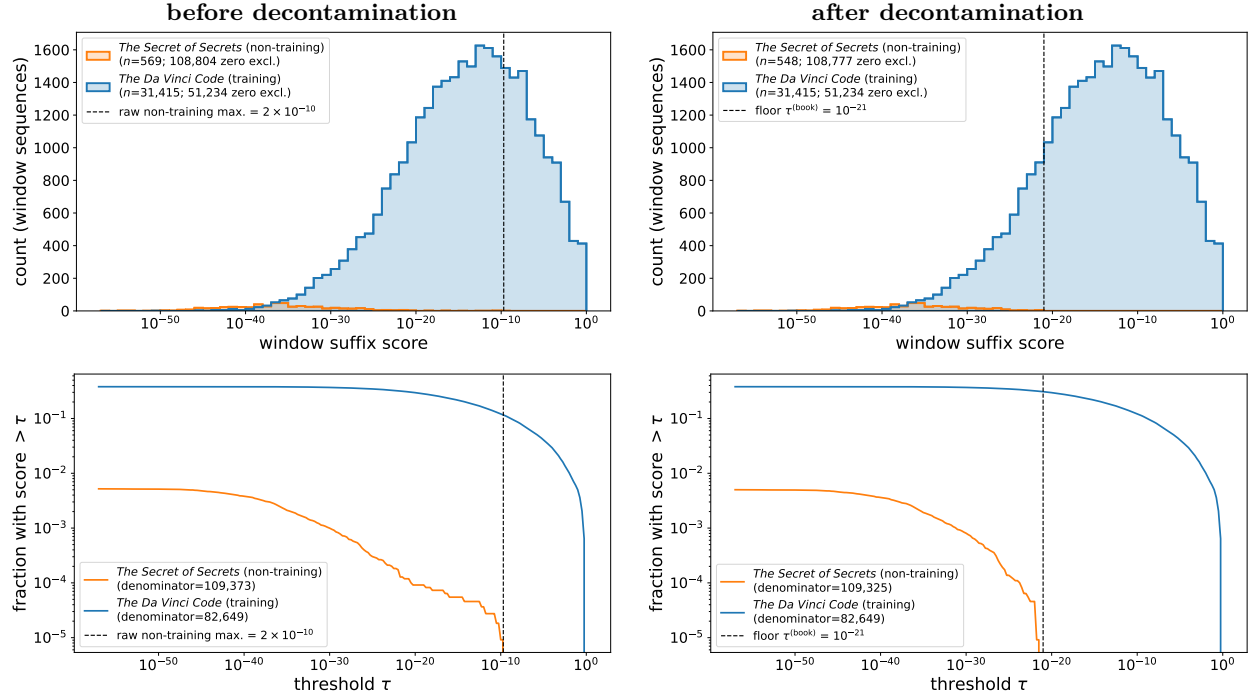


Training text	70B $\tau^{(book)}$		8B $\tau^{(book)}$	
	$a=10$	$a=50$	$a=10$	$a=50$
<i>The Da Vinci Code</i> (Brown [3])	0.238	10^{-21}	0.221	10^{-22}
<i>We Were Eight Years in Power</i> (Coates [12])	0.630	10^{-24}	0.384	10^{-36}
<i>The Hunger Games</i> (Collins [14])	0.795	10^{-27}	0.257	10^{-31}
<i>Killer Instinct</i> (Finder [22])	0.111	10^{-24}	0.061	10^{-26}

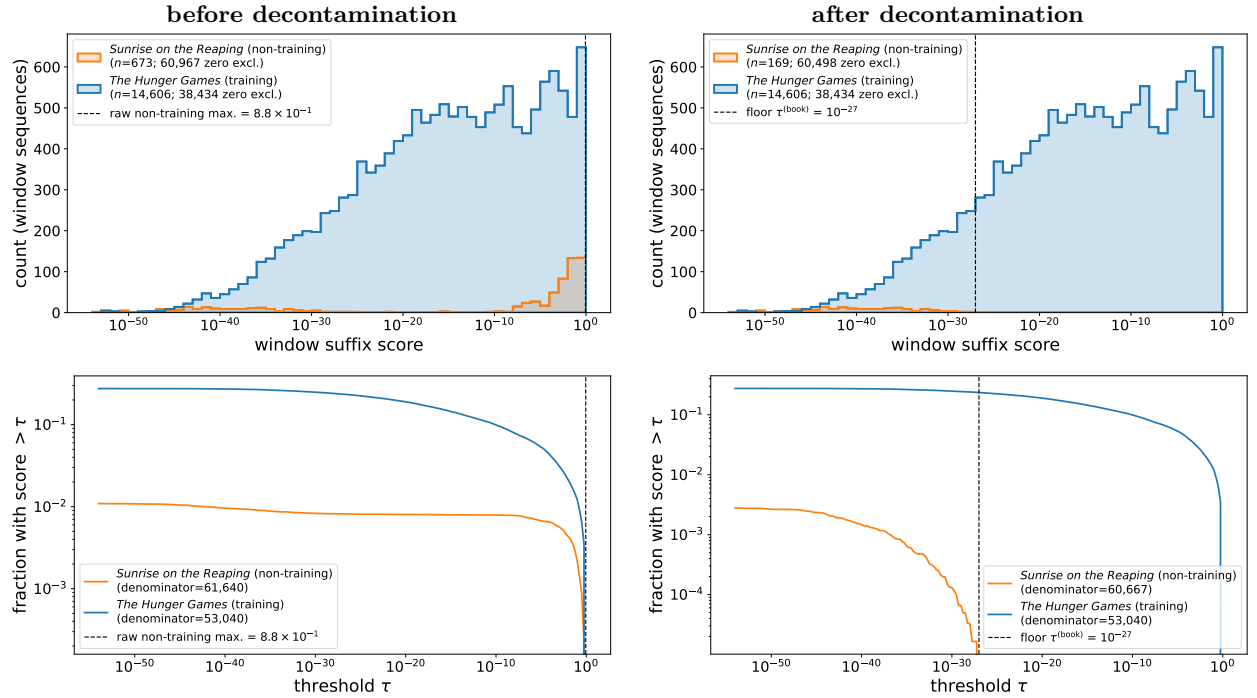
Figure 15: Left: Coverage and non-training floor probabilities for author-book pairs for Llama 3.1 70B (50-token suffixes). We do not plot results for *Killer Instinct* [22], as it isn’t memorized. Right: Showing how the calibrated floor $\tau^{(book)}$ varies across prefix length ($a \in \{10, 50\}$) and model size (70B and 8B) for Llama 3.1 70B.

B.1 Decontaminating nulls

We developed an agent skill for assisting with the removal of contamination (actual members) from books that were not included in Llama 3.1’s training data. We plot the results of this process: for a given pair of books, the distribution over sequence probabilities for both both the in-training and non-training books both before decontamination and after.

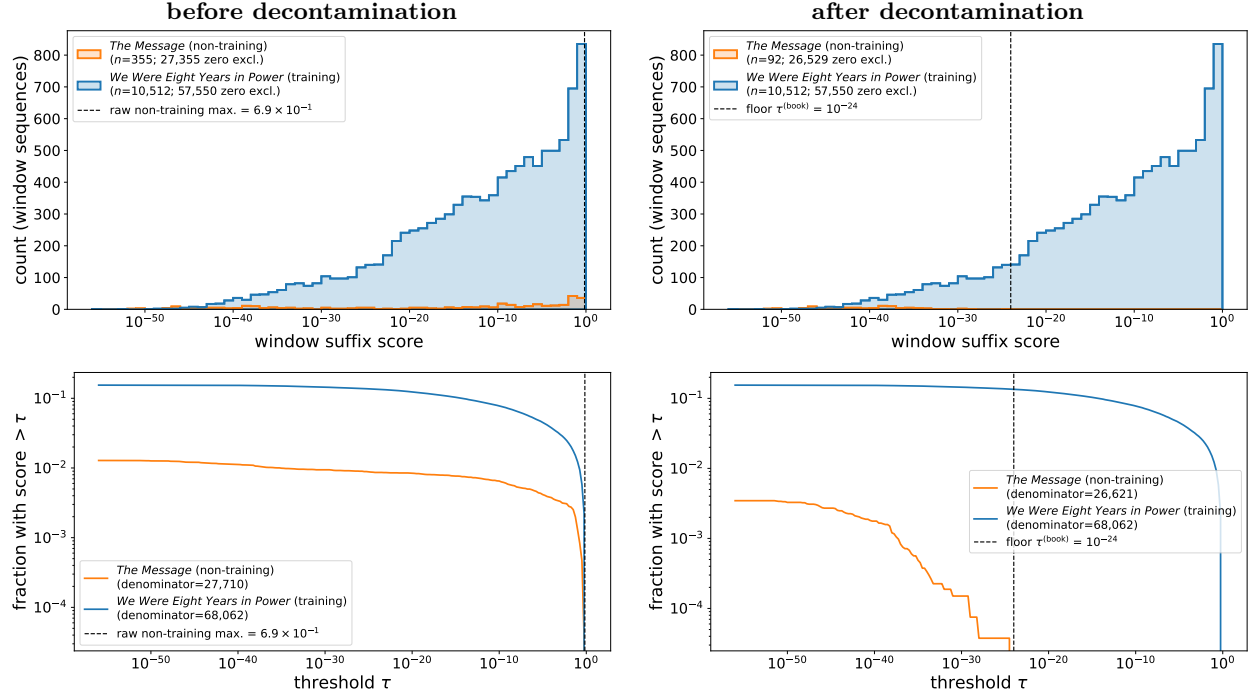


(a) Dan Brown: *The Secret of Secrets* [4] (non-training book) and *The Da Vinci Code* [3] (in-training book).

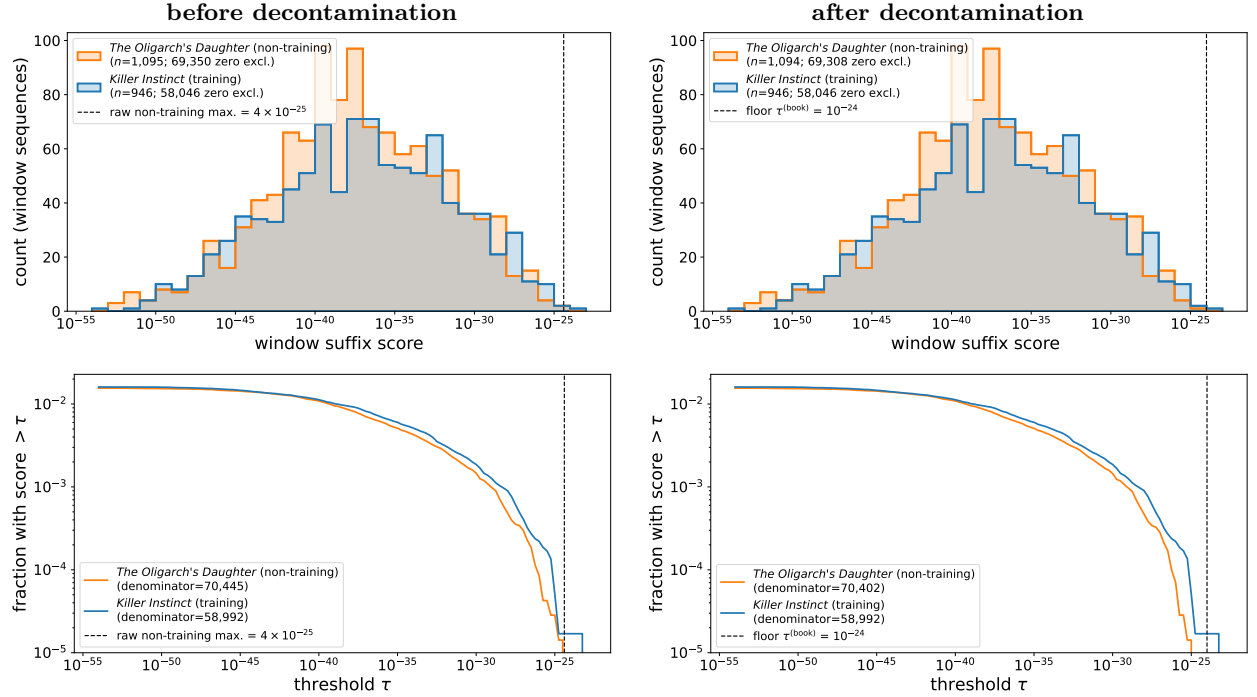


(b) Suzanne Collins: *Sunrise on the Reaping* [15] (non-training book) and *The Hunger Games* [14] (in-training book).

Figure 16: Showing the results of decontamination (and the resulting $\tau^{(\text{book})}$) for Dan Brown and Suzanne Collins.



(a) Ta-Nehisi Coates: *The Message* [13] (non-training book) and *We Were Eight Years in Power* [12] (in-training book).



(b) Joseph Finder: *The Oligarch's Daughter* [23] (non-training book) and *Killer Instinct* [22] (in-training book).

Figure 17: Showing the results of decontamination (and the resulting $\tau^{(\text{book})}$) for Ta-Nehisi Coates and Joseph Finder.

C Extractable memorization

Fix a single prompt $\mathbf{p}$, and let

$$p_{\mathbf{s}} := q_{\theta, \phi}(\mathbf{s} \mid \mathbf{p})$$

be the per-query generation probability of $\mathbf{s}$ given $\mathbf{p}$.

An attacker who issues n independent queries fails on all of them with probability $(1 - p_{\mathbf{s}})^n$, so the probability of generating the candidate $\mathbf{s}$ at least once is

$$\Pr_{\text{gen}}(n) = 1 - (1 - p_{\mathbf{s}})^n. \quad (12)$$

Budget for a target confidence. Fix a confidence level $c \in (0, 1)$: the probability required of at least one successful generation of $\mathbf{s}$. The smallest number of queries n with $\Pr_{\text{gen}}(n) \geq c$:

$$\begin{aligned} 1 - (1 - p_{\mathbf{s}})^n \geq c &\iff (1 - p_{\mathbf{s}})^n \leq 1 - c \\ &\iff n \log(1 - p_{\mathbf{s}}) \leq \log(1 - c) \\ &\iff n \geq \frac{\log(1 - c)}{\log(1 - p_{\mathbf{s}})}, \end{aligned} \quad (13)$$

where the inequality flips in the last line because $\log(1 - p_{\mathbf{s}}) < 0$ for $p_{\mathbf{s}} \in (0, 1)$. Both logs are negative, so the ratio is positive; the required budget is

$$n(p_{\mathbf{s}}; c) = \left\lceil \frac{\log(1 - c)}{\log(1 - p_{\mathbf{s}})} \right\rceil. \quad (14)$$

It grows without bound as $p_{\mathbf{s}} \rightarrow 0$ (rarer sequences cost more queries) and shrinks as $c \rightarrow 0$.

Inverse: probability reachable at a fixed budget. Solving Equation 12 for $p_{\mathbf{s}}$ at fixed n gives the smallest per-query probability that reaches confidence c within n queries:

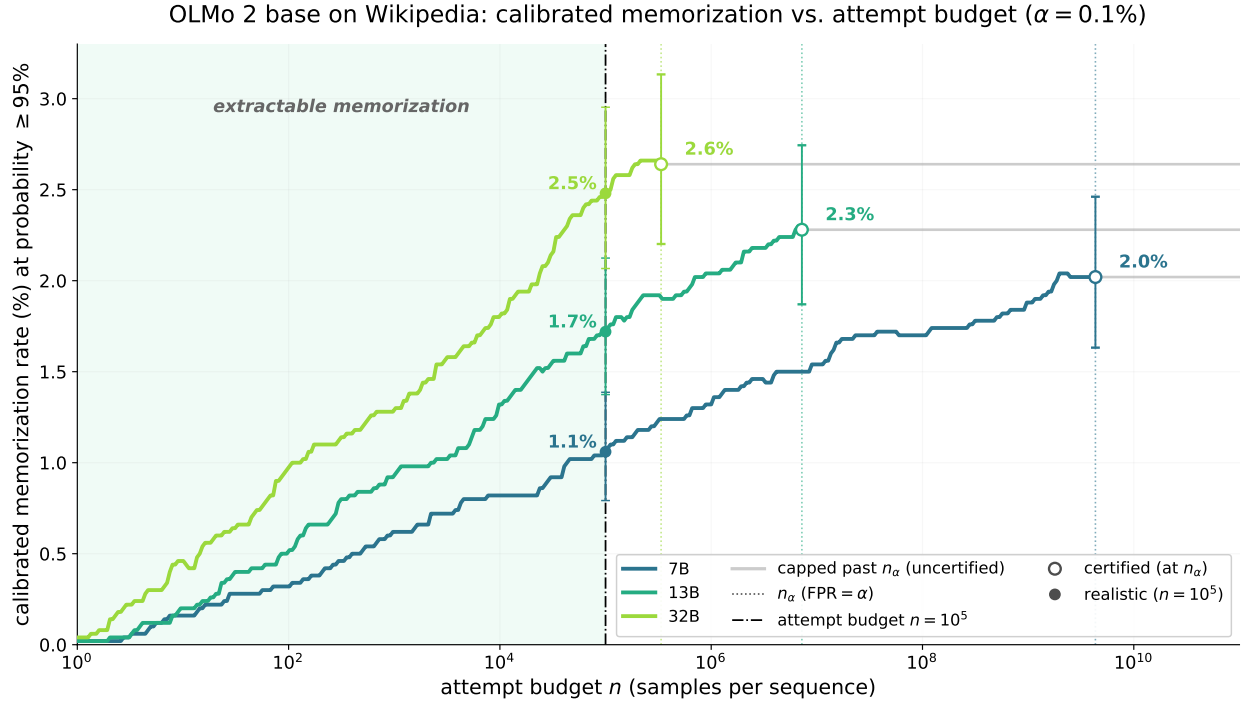
$$p_{\mathbf{s}}(n; c) = 1 - (1 - c)^{1/n}. \quad (15)$$

This is the map used to place the vertical budget lines: a window scoring $\geq p_{\mathbf{s}}(n; c)$ is extractable within n queries at confidence c (e.g., the $n = 10^5$ line uses $p_{\mathbf{s}}(10^5; 0.95)$).

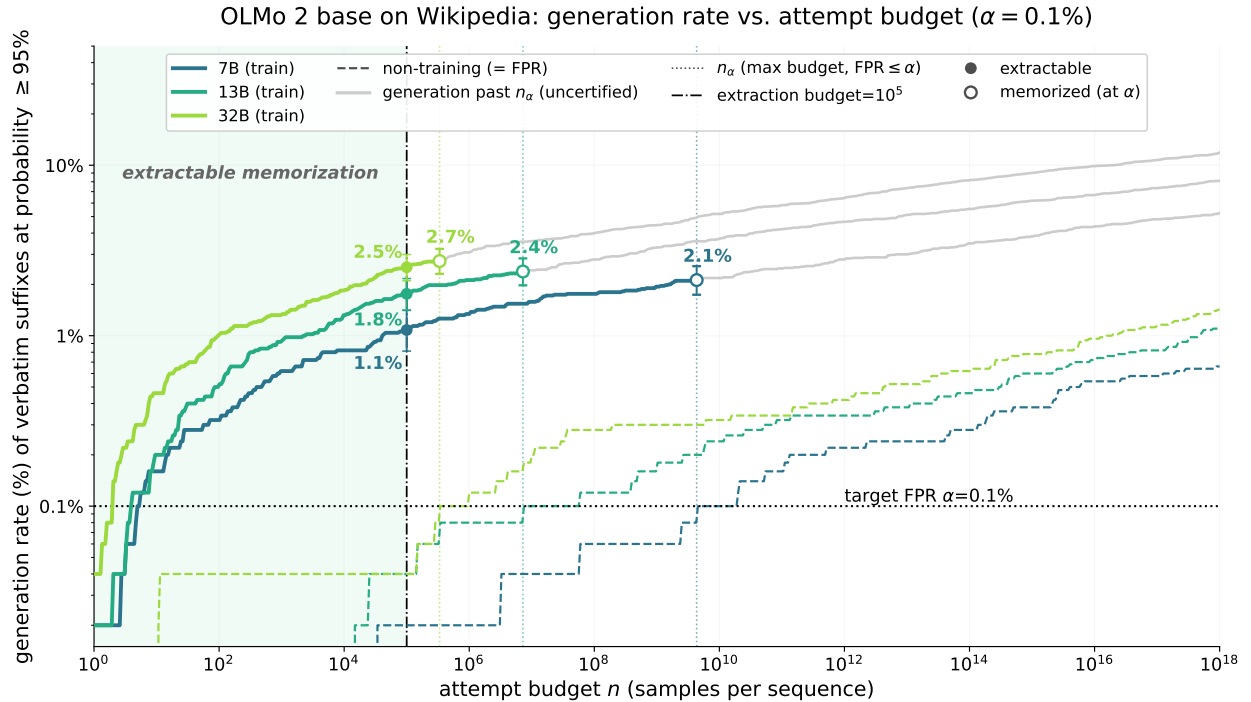
Small- $p_{\mathbf{s}}$ approximation. For small $p_{\mathbf{s}}$, $\log(1 - p_{\mathbf{s}}) \approx -p_{\mathbf{s}}$, so

$$n(p_{\mathbf{s}}; c) \approx \frac{-\log(1 - c)}{p_{\mathbf{s}}} \quad (16)$$

At $c = 0.95$, $-\log(0.05) \approx 2.996$, giving $n \approx 3/p_{\mathbf{s}}$: reaching 95% confidence costs roughly three queries per unit of inverse extraction probability.

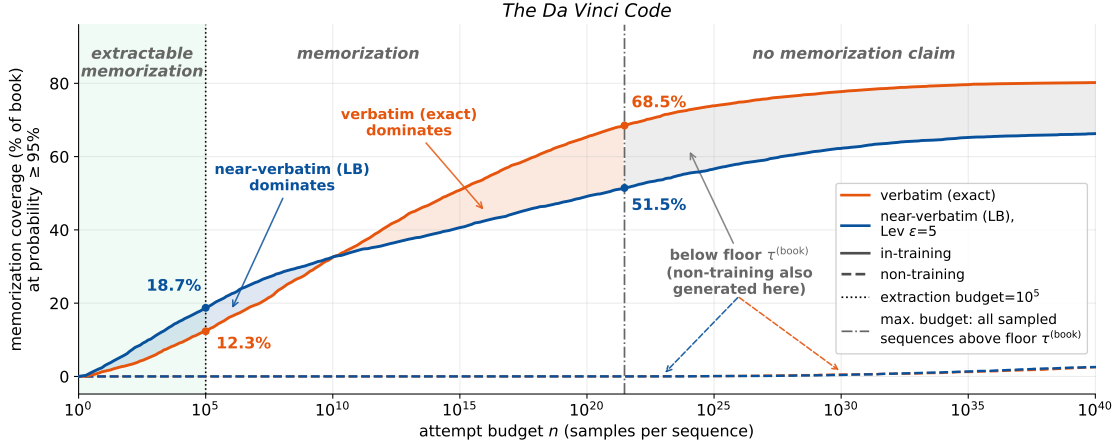


(a) At fixed FPR $\alpha = 0.1\%$, we plot the calibrated memorization rate (at confidence $c \geq 95\%$) for a given attempt budget n (independent samples per sequence) for OLMo 2 models on Wikipedia.

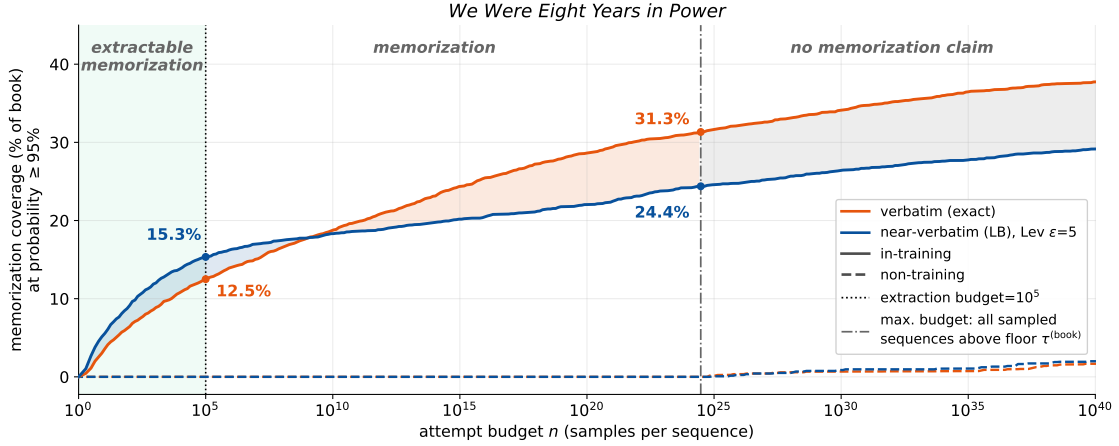


(b) We plot the verbatim generation rate (at confidence $c \geq 95\%$) for a given attempt budget n (independent samples per sequence) for OLMo 2 models on Wikipedia. We show rates for both training and non-training data for each model, and plot a reference line for target FPR $\alpha = 0.1\%$. Each point on training-data curves indicates the attempt budget n where the rate of generating non-training data hits the target FPR α .

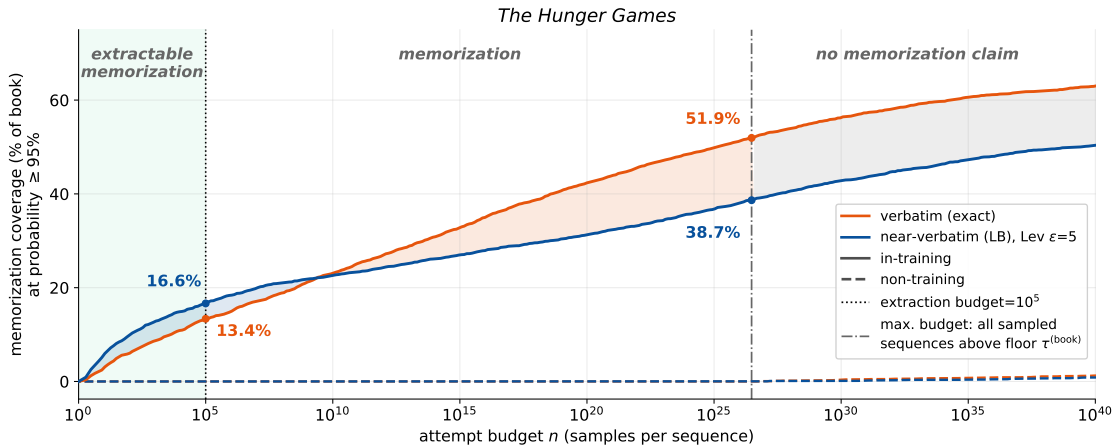
Figure 18: Two budget-level views of OLMo 2 models on Wikipedia.



(a) Dan Brown. Both verbatim and near-verbatim probabilistic extraction yield the same $\tau^{(\text{book})} = 10^{-21}$.



(b) Ta-Nehisi Coates. Both verbatim and near-verbatim probabilistic extraction yield the same $\tau^{(\text{book})} = 10^{-24}$.



(c) Suzanne Collins. The two extraction procedures yield different floors. For verbatim, $\tau^{(\text{book})} = 10^{-27}$. For near-verbatim (the k -CBS lower-bound estimate), $\tau^{(\text{book})} = 10^{-26}$. It is simpler to illustrate this plot with a single $\tau^{(\text{book})}$. To be conservative, we use the higher $\tau^{(\text{book})} = 10^{-26}$ (near-verbatim). Note that the verbatim above-floor coverage numbers are slightly higher; please refer to the other plots (bar and margin).

Figure 19: Memorization coverage, with reference lines for non-training data (which exceed 0) after the maximum sampling budget for the respective floor, $\tau^{(\text{book})}$.

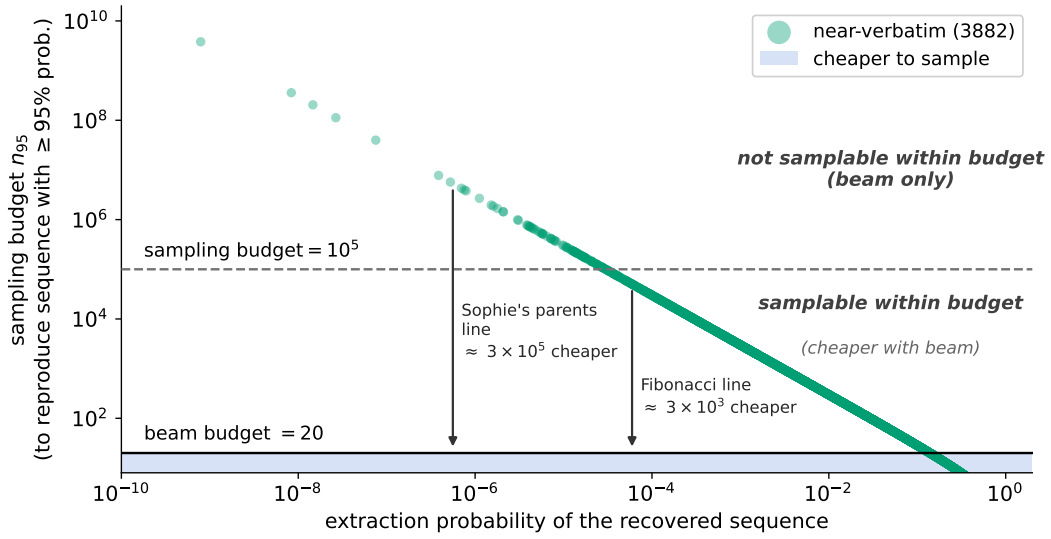


Figure 20: Illustrating the cost savings of using k -CBS to deterministically generate near-verbatim training data from *The Da Vinci Code*, rather than sampling.